



第2章 递归与分治策略





学习要点:

- 理解递归的概念（**Fibonacci**数列、 **Hanoi**塔问题）
- 掌握设计有效算法的分治策略
- 掌握范例的设计技巧。

分治策略设计范例

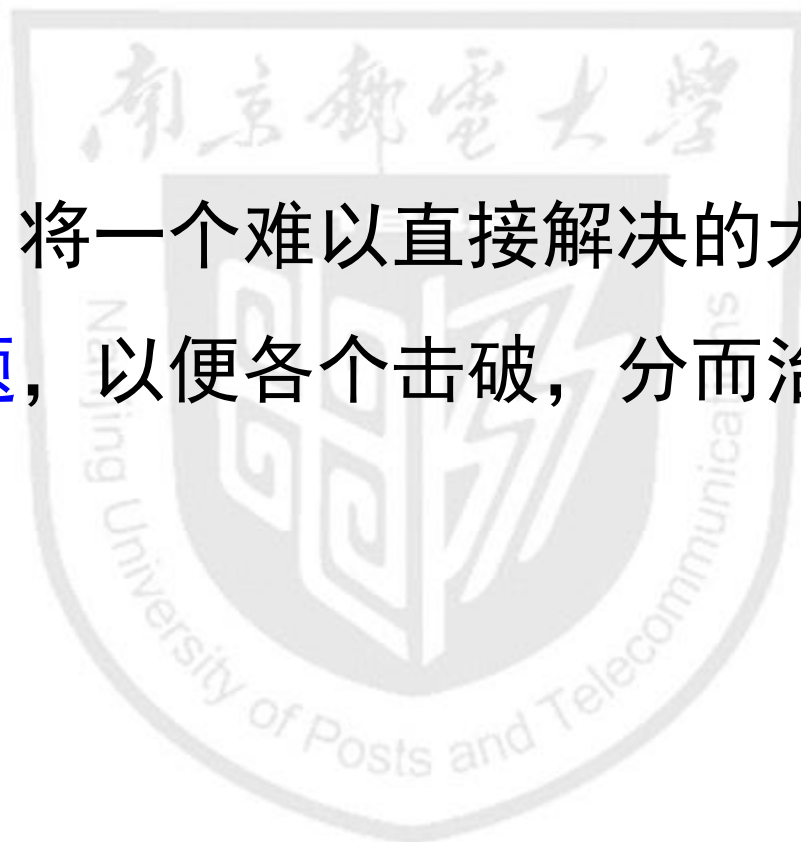
- (1) 二分搜索技术
- (2) 大整数乘法
- (3) 合并排序
- (4) 快速排序



算法总体思想

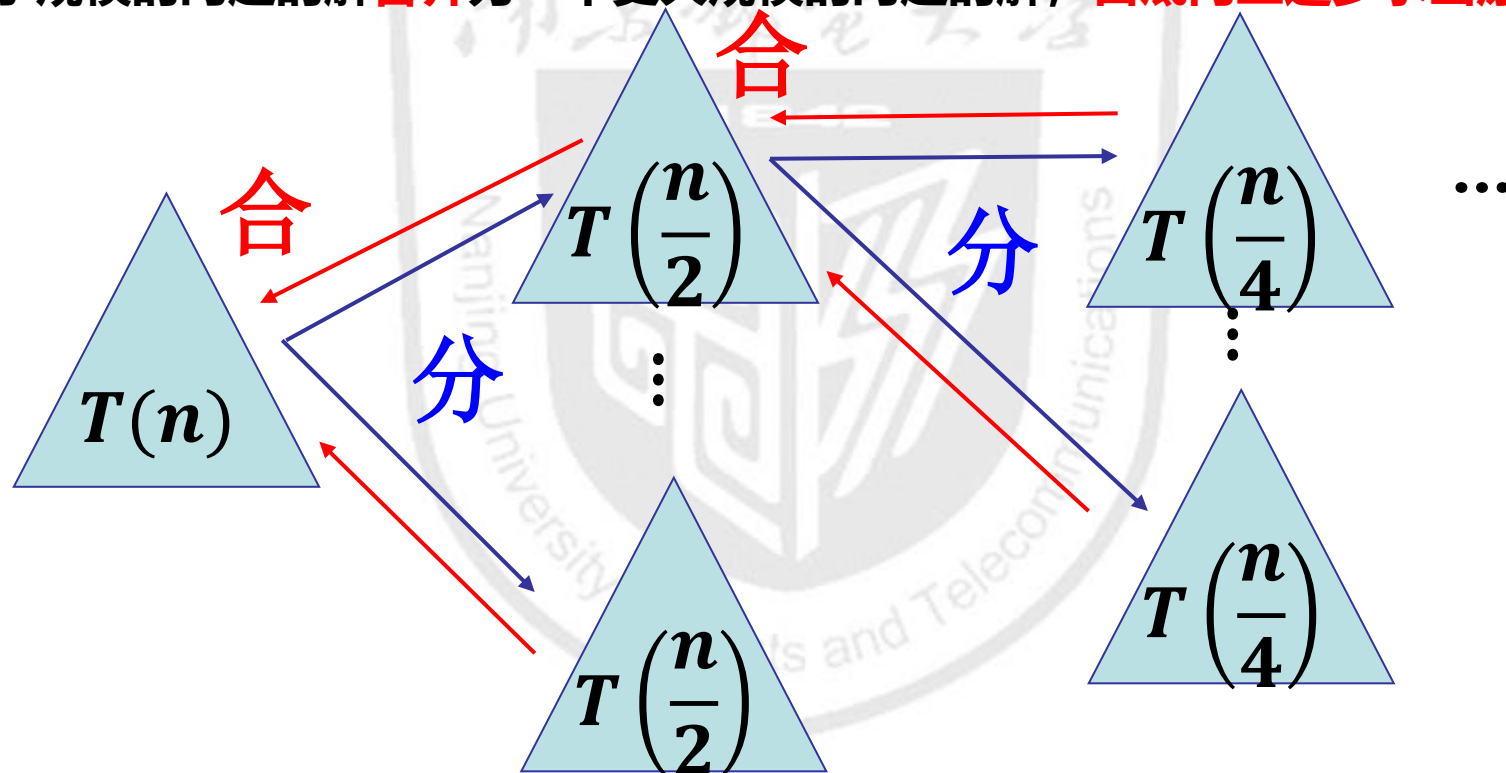
凡治众如治寡，分数是也。——孙子兵法

分治法的设计思想：将一个难以直接解决的大问题，分割成一些规模较小的相同问题，以便各个击破，分而治之。



算法总体思想

- (1) 将要求解的较大规模的问题**分割成** k **个更小规模的子问题**;
- (2) 分别求解这 k 个子问题。如果子问题的规模仍然不够小, 则**再划分为** k **个子问题**, 如此递归的进行下去;
- (3) 分解终止条件: **问题规模足够小, 容易求解**。
- (4) 将求出的小规模的问题的解**合并**为一个更大规模的问题的解, **自底向上逐步求出原来问题的解**。



递归的概念

由分治法产生的子问题往往是原问题的缩减模式（问题相同，规模变小），为使用递归技术提供了方便。

递归算法：直接或间接地调用自身的算法。

递归函数：用函数自身给出定义的函数。

递归过程：反复应用分治手段，可使子问题与原问题类型一致而规模却不断缩小，最终使子问题缩小到可直接求解。

分治与递归像一对孪生兄弟

递归的例子

例1 阶乘函数

阶乘函数可递归地定义为：

$$n! = \begin{cases} 1, & n = 0 \\ n(n-1)!, & n > 0 \end{cases}$$

边界条件
(非常重要)

递归方程

边界条件与**递归方程**是递归函数的**两个要素**，递归函数只有具备了这两个要素，才能在有限次计算后得出结果。

例1 阶乘函数（程序）

```
int factorial (int n)
{
    if ( n == 0)
        return 1;
    return n * factorial(n-1);
}
```

边界条件

递归方程

$$n! = \begin{cases} 1, & n = 0 \\ n(n-1)!, & n > 0 \end{cases}$$

例2 Fibonacci数列

无穷数列 {1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ……} 称为Fibonacci数列，可递归地定义为：

$$F(n) = \begin{cases} 1 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

边界条件

递归方程

程序：

```
int fibonacci(int n)
{
    if (n <= 1) return 1;
    return fibonacci(n-1) + fibonacci(n-2);
}
```

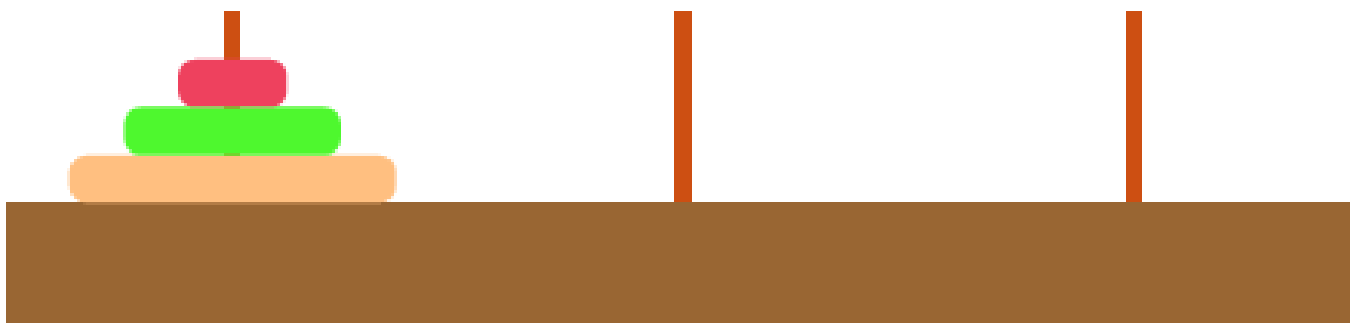

例3 Hanoi 塔问题

场景： 设A, B, C是3个塔座。开始时，在塔座A上有一叠共 n 个圆盘，这些圆盘自下而上，由大到小地叠在一起。各圆盘从小到大编号为 $1, 2, \dots, n$ 。

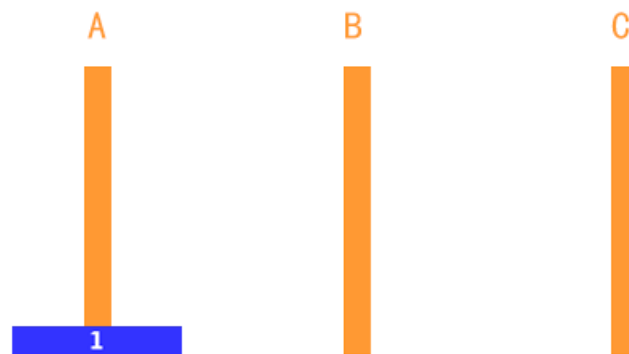
要求： 将塔座A上的这一叠圆盘移到塔座C上，并按同样顺序叠置。

规则：

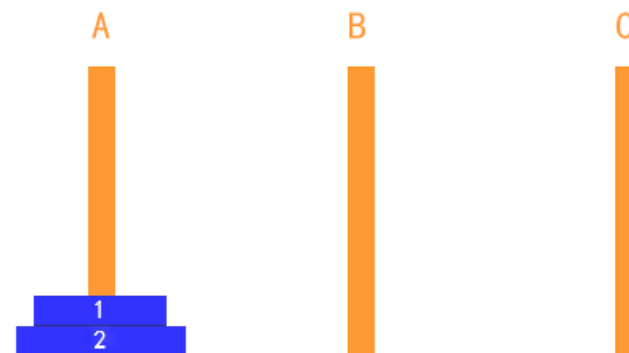
- (1) 每次只能移动1个圆盘；
- (2) 任何时刻都不允许将较大的圆盘压在较小的圆盘之上；
- (3) 在满足移动规则1和2的前提下，可将圆盘移至A, B, C中任一塔座上。



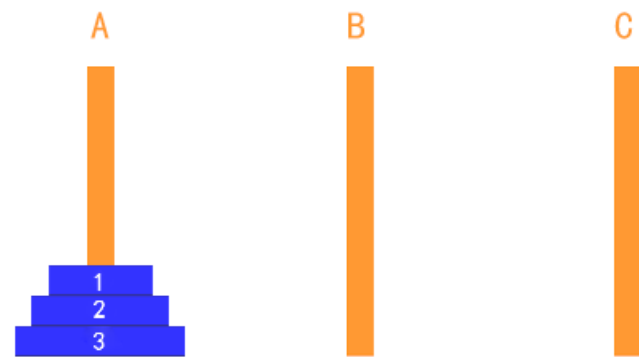
1个盘子



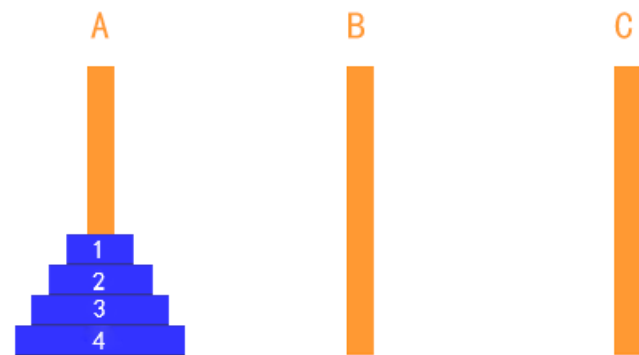
2个盘子



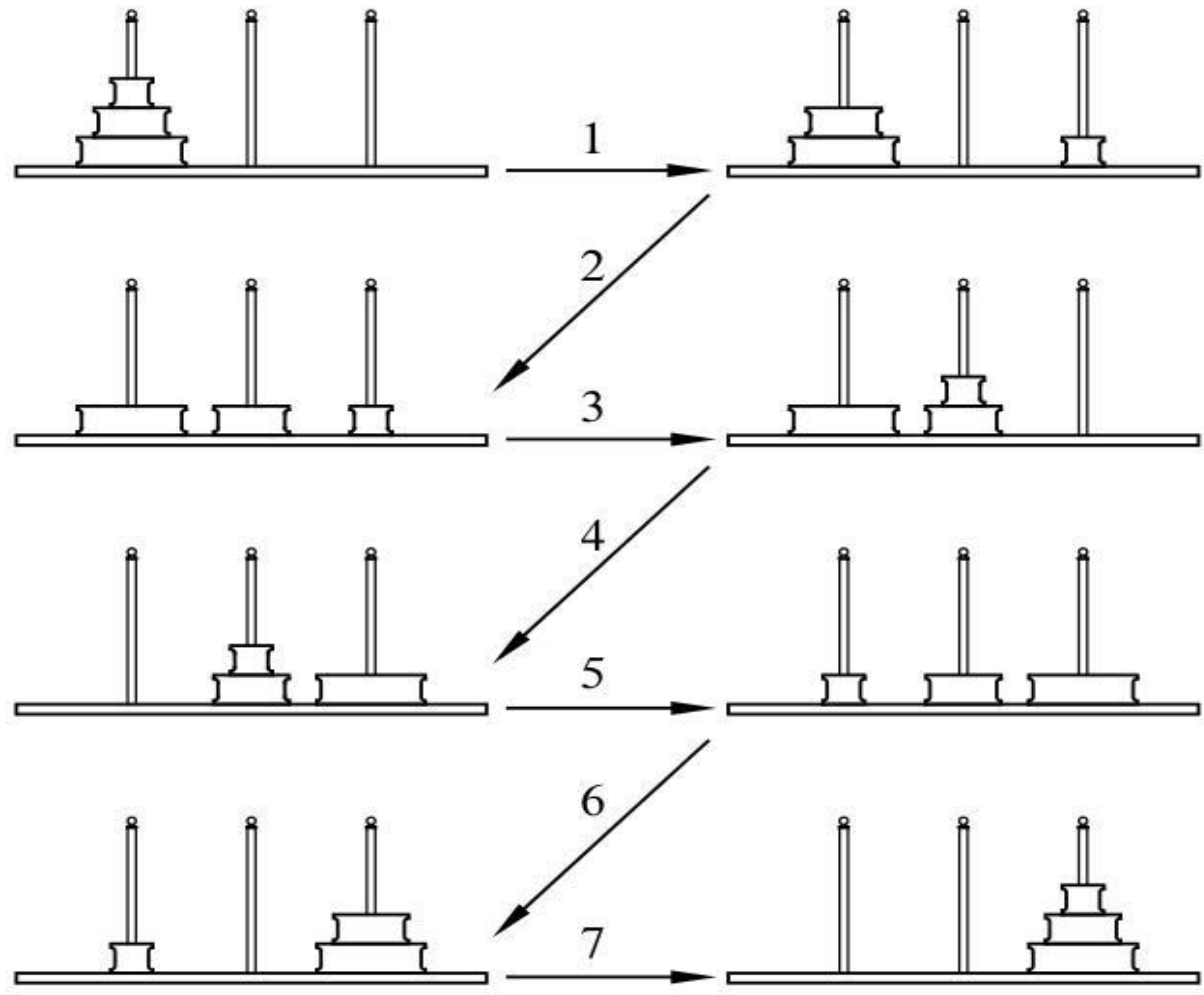
3个盘子



n个盘子



分步展示

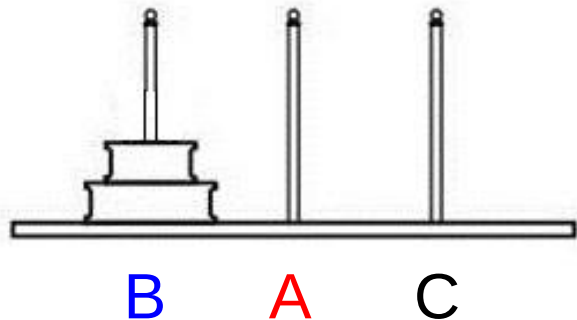


如何体现递归？

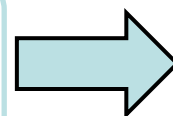
问题分析

- (1) 把 $n - 1$ 号盘子移动到缓冲区
- (2) 把 n 号从起点移到终点
- (3) 然后把缓冲区的 $n - 1$ 号盘子也移到终点

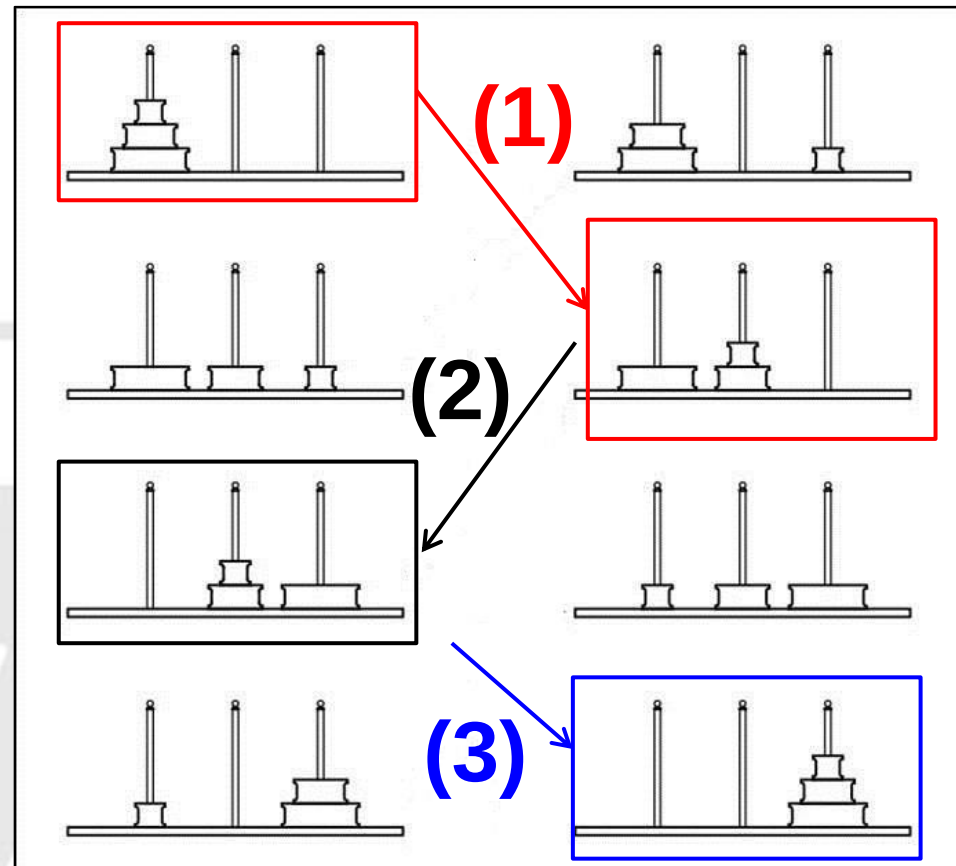
第(3)步中, 如将B作为初始柱, A作为缓存柱
(即交换A、B柱), 同时忽略C柱上的3号盘



Hanoi问题由3个盘子化简为
2个盘子的相同问题



递归



程序

```
void hanoi(int n, int a, int b, int c)
```

```
{  
  if (n > 0)  
  {  
    hanoi(n-1, a, c, b);  
    move(a,c);  
    hanoi(n-1, b, a, c);  
  }  
}
```

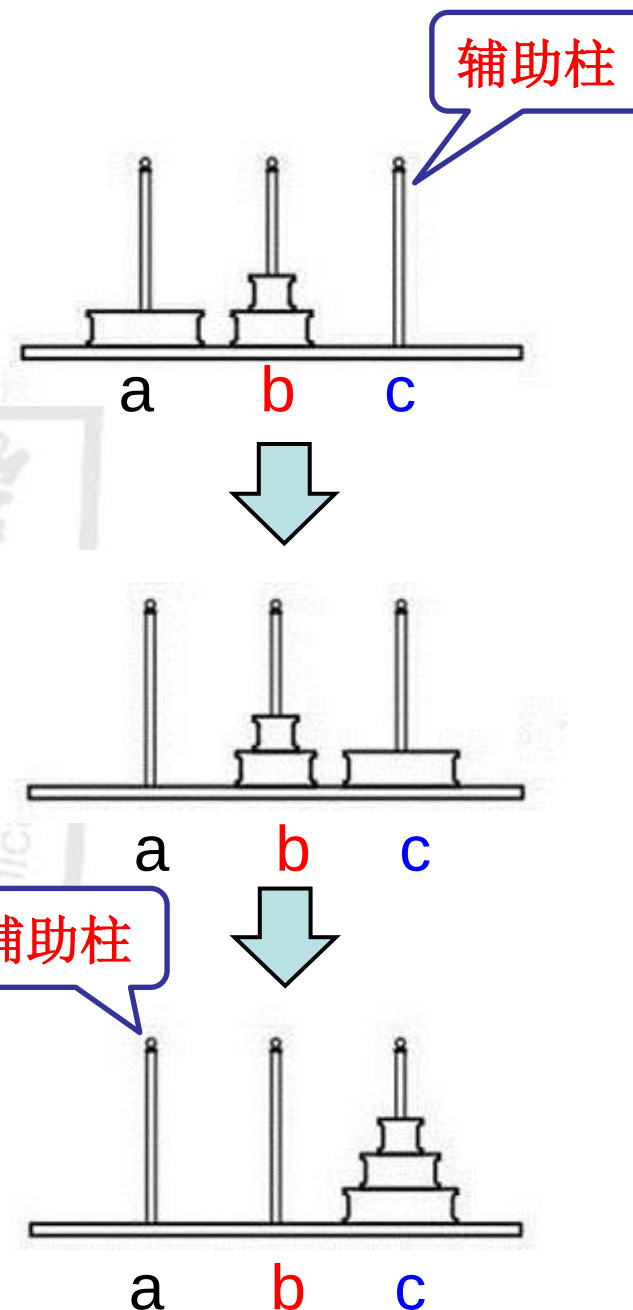
以**c**为辅助，将把 $n - 1$ 个盘子移动到**b**

将**a**上的碟子移至**c**

以**a**为辅助，将把 $n - 1$ 个盘子移动到**c**

hanoi(n, a, b, c): 以**b**柱为辅助，将**a**柱的 n 个碟子以相同的上下顺序移至**c**柱

move(a, b): 将**a**柱上编号为 n 的碟子移至**b**柱



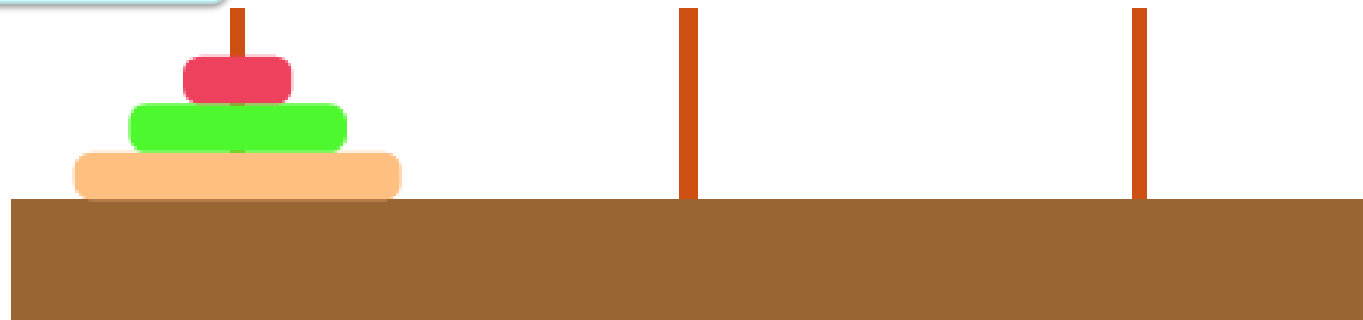
最少需要移动多少步完成Hanoi塔问题？



步骤：

- (1) 把 $n - 1$ 号盘子移动到缓冲区
- (2) 把 n 号从起点移到终点
- (3) 然后把缓冲区的 $n - 1$ 号盘子也移到终点

设移动 n 个圆盘所需的最少步数为 $F(n)$



步骤:

- | | |
|------------------------------|---------------|
| (1) 把 $n - 1$ 号盘子移动到缓冲区 | 步数 $F(n - 1)$ |
| (2) 把 n 号从起点移到终点 | 1 |
| (3) 然后把缓冲区的 $n - 1$ 号盘子也移到终点 | $F(n - 1)$ |

递推公式:
$$F(n) = \begin{cases} 2F(n - 1) + 1, & n > 1 \\ 1, & n = 1 \end{cases}$$



递推公式: $F(n) = \begin{cases} 2F(n-1) + 1, & n > 1 \\ 1, & n = 1 \end{cases}$

$$\begin{aligned} F(n) &= 2F(n-1) + 1 = 2(2F(n-2) + 1) + 1 \\ &= 2^2F(n-2) + 2 + 1 = 2^2F(n-2) + 2^1 + 2^0 \\ &= 2^3F(n-3) + 4 + 2 + 1 = 2^3F(n-3) + 2^2 + 2^1 + 2^0 \\ &\dots \\ &= 2^kF(n-k) + 2^{k-1} + \dots + 2^1 + 2^0 \\ &\dots \\ &= 2^{n-1}F(n-(n-1)) + 2^{n-2} + \dots + 2^1 + 2^0 \\ &= 2^{n-1} + 2^{n-2} + \dots + 2^1 + 2^0 \\ &= 2^n - 1 \quad (\text{等比数列求和}) \end{aligned}$$

└─ 步数: $F(n) = 2^n - 1$ $\Longrightarrow$ 复杂度: $O(2^n)$

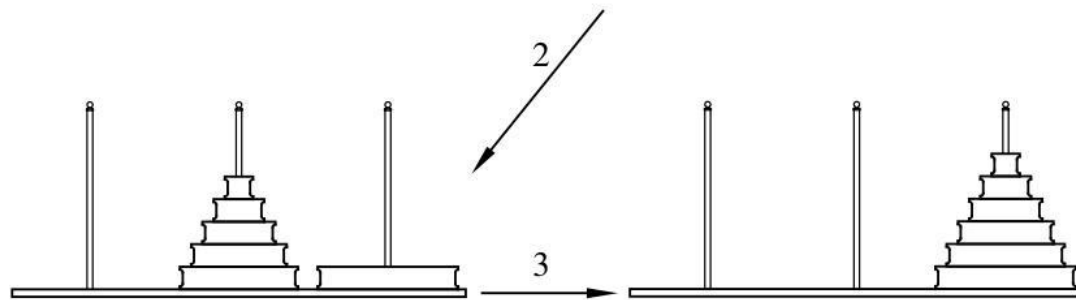


Hanoi 塔问题拓展

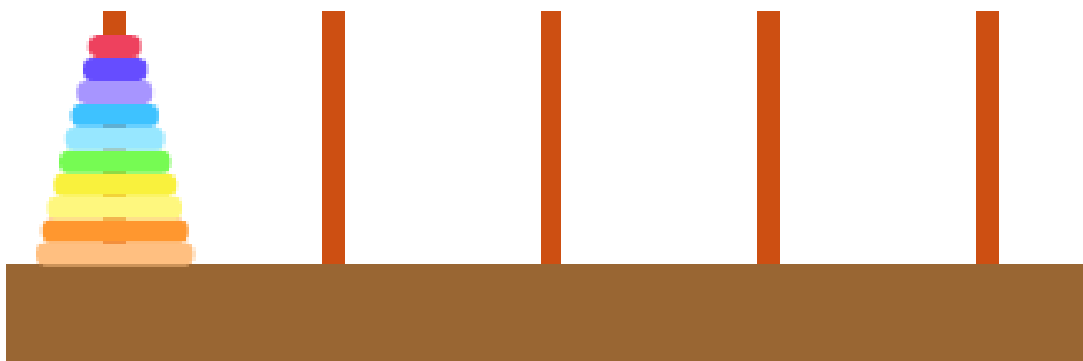
3个柱
 N 个盘

扩展

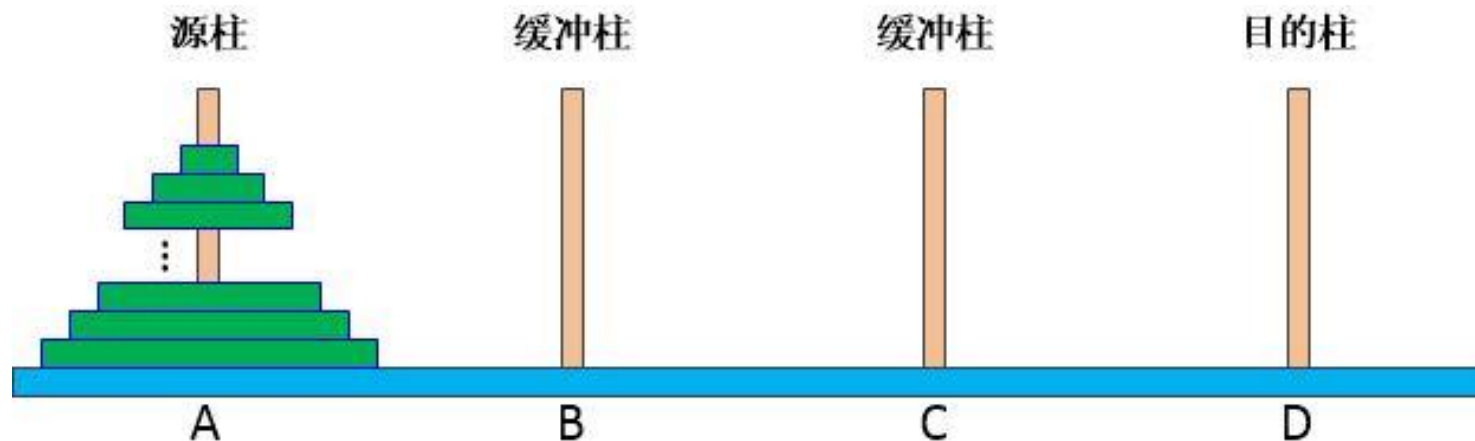
M 个柱
 N 个盘



多柱Hanoi 塔问题

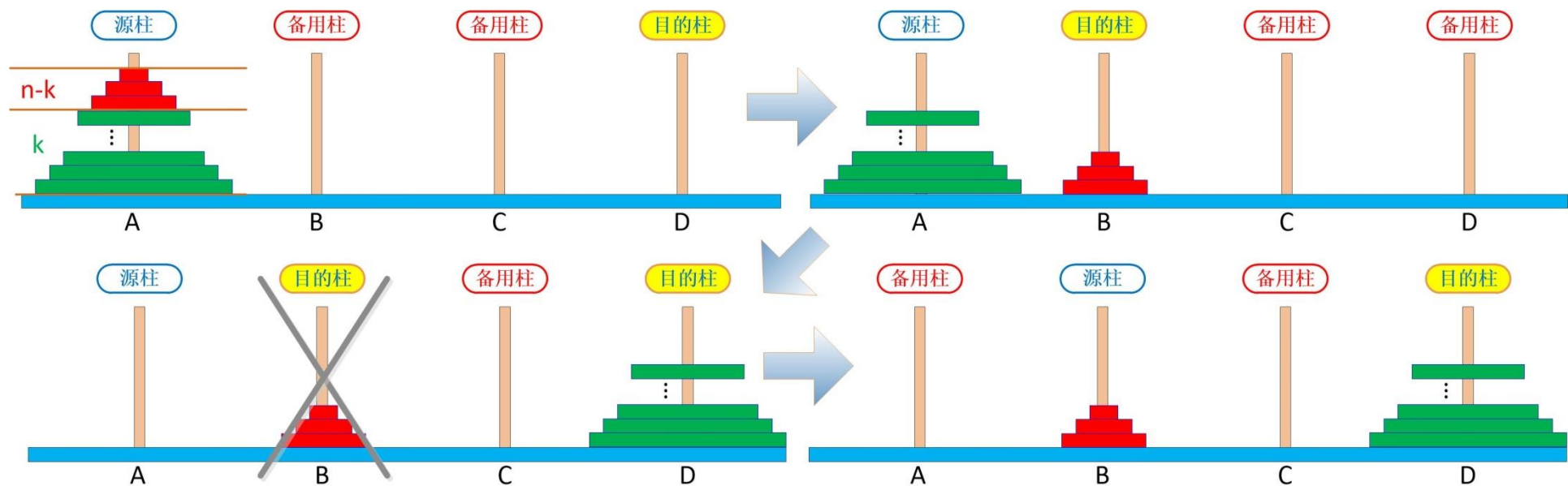


4柱Hanoi塔



问题:

- 1) 如何体现递归?
- 2) 可否借鉴3柱Hanoi塔问题的思路? (B/C只用其中一个?)
- 3) 最少步数及复杂度?



步骤:

- 1) 将A柱上 n 个盘子划分为上下两部分，下方部分共有 $k(1 \leq k \leq n)$ 个盘子，上方部分共有 $n - k$ 个盘子；
- 2) 将A柱上面部分 $n - k$ 个盘子使用4柱算法经过C、D柱移至B柱；
- 3) 将A柱剩余的 k 个盘子使用3柱算法经过C柱移至D柱；
- 4) 将B柱上的 $n - k$ 个盘子使用4柱算法经过A、C柱移至D柱。

步数

$$\begin{aligned}
 &F(n - k) \\
 &2^k - 1 \\
 &F(n - k)
 \end{aligned}$$

$$\text{递推公式: } F(n) = \begin{cases} \min_{1 \leq k \leq n} \{2F(n - k) + 2^k - 1\}, & n > 1 \\ 1, & n = 1 \end{cases}$$



递推公式:
$$F(n) = \begin{cases} \min_{1 \leq k \leq n} \{2F(n-k) + 2^k - 1\}, & n > 1 \\ 1, & n = 1 \end{cases}$$



k 取多少最合适?



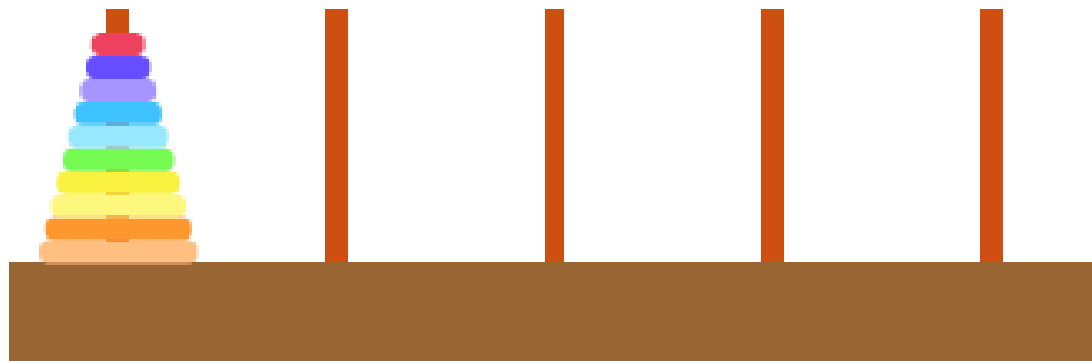
最佳 **k** :
$$k = \left\lfloor \frac{\sqrt{8n+1}-1}{2} \right\rfloor$$

$$F(n) = \left[n - \frac{k^2 - k + 2}{2} \right] 2^k + 1$$



复杂度:
$$O(\sqrt{n} 2^{\sqrt{n}})$$

m 柱Hanoi塔问题 ($m \geq 5$)



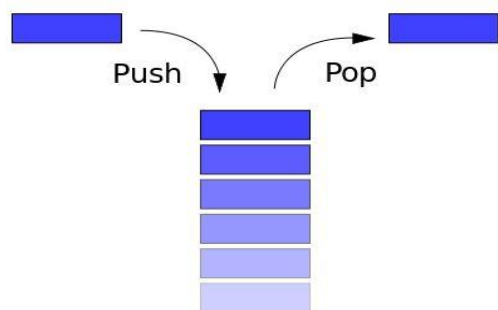
问题:

- (1) 可否借鉴4柱Hanoi塔问题的思路?
- (2) 最少步数及复杂度?
- (3) 复杂度是否随 m 的增加而增加?

留给大家开放式思考

递归的实现

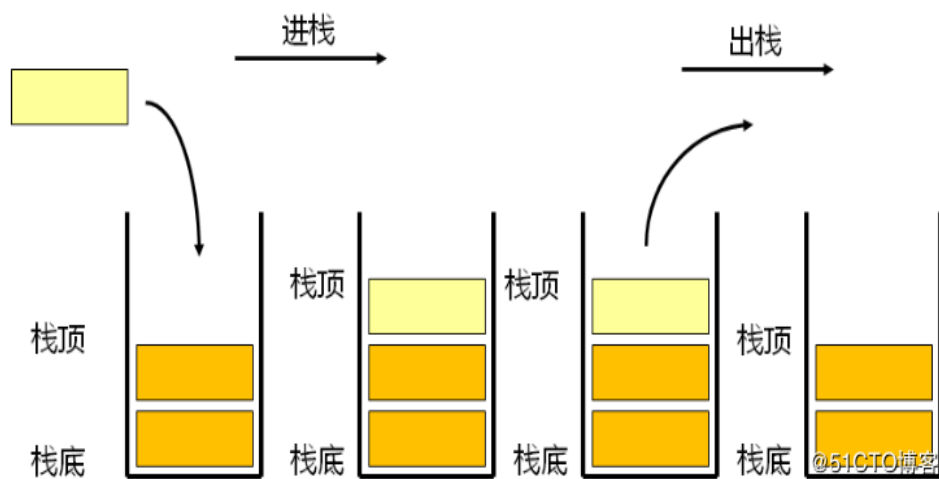
栈



入栈：算法调用另一个算法

- 1、将所有的实参、返回地址等信息传递给被调算法；
- 2、为被调函数的局部变量分配存储区；
- 3、将控制转移到被调算法入口。

特点：后进先出



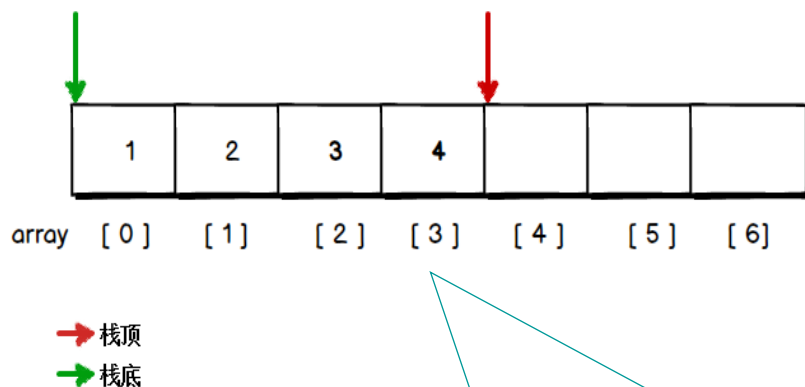
出栈：被调用算法返回调用算法

- 1、保存被调算法的计算结果；
- 2、释放被调算法的数据区；
- 3、依照被调算法保存的返回地址将控制转移到调用算法。

栈的实现

数组形式

基于数组的栈

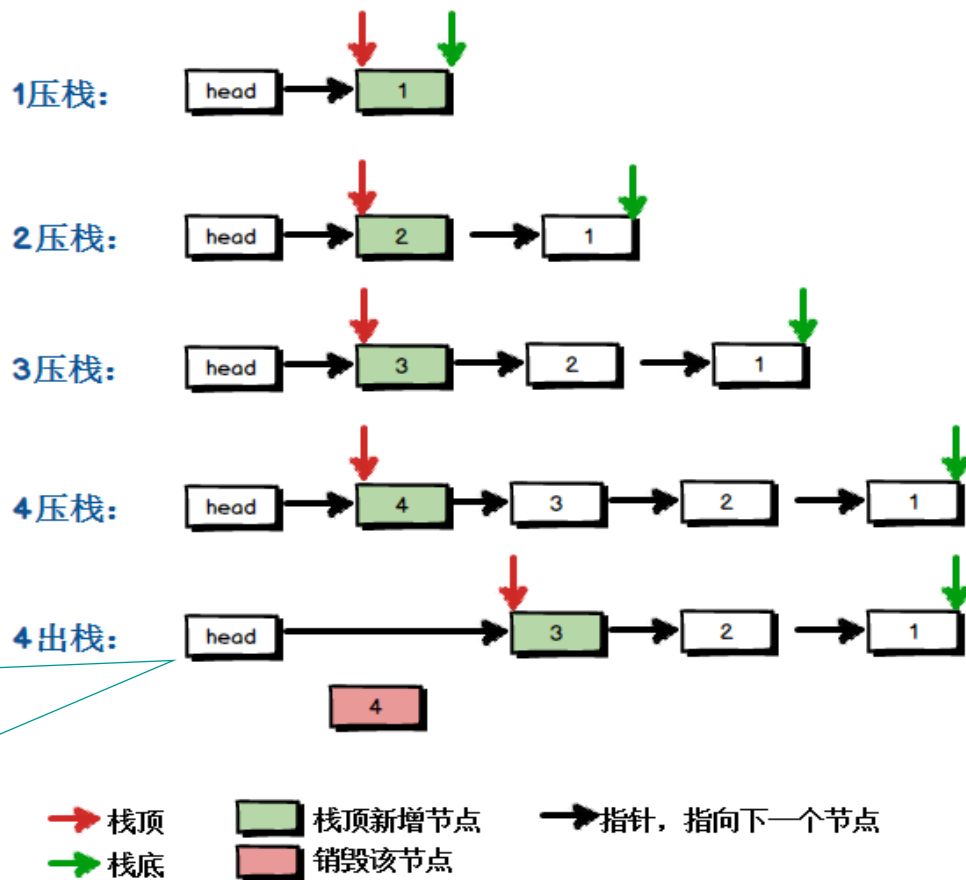


通常以数组头为栈底，数组头到数组尾为栈顶的生长方向

通常以链表头为作为栈顶较为合适，方便节点的插入与删除。压栈产生的新节点将一直出现在链表的头部

链表形式

基于单链表的栈



递归小结

优点：

- 结构清晰，可读性强
- 容易用数学归纳法来证明算法的正确性，便于算法设计、程序调试

缺点：

- 运行效率较低（进出栈对被调用算法的控制）
- 耗费的计算时间、占用的存储空间均比非递归算法要多



解决方法：在递归算法中消除递归调用，使其转化为非递归算法

1、采用一个用户定义的栈来模拟系统的递归调用工作栈。该方法通用性强，但本质上还是递归，只不过人工做了本来由编译器做的事情，优化效果不明显。

2、用递推来实现递归函数。

3、通过变换能将一些递归转化为尾递归，从而迭代求出结果。

后两种方法在时空复杂度上均有较大改善，但其适用范围有限。

非递归的例子

非递归方式:

阶乘 $n! = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n$

实现: for 循环

Fibonacci 数列

$$F(n) = \frac{1}{5} \left(\left(\frac{1 + \sqrt{5}}{2} \right)^{n+1} - \left(\frac{1 - \sqrt{5}}{2} \right)^{n+1} \right)$$

实现: 直接计算

分治策略框架程序

```
divide-and-conquer(P){
```

- (1) if ($|P| \leq n_0$) adhoc(P); //递归出口, 用特定程序解决基础问题
- (2) **divide** P into smaller subinstances **P1,P2,...,Pk**; //分解出子问题
- (3) for ($i = 1, i \leq k, i++$)
 $y_i = \text{divide-and-conquer}(P_i)$; //递归求解各子问题
- (4) return merge($y_1, \dots, y_k$); //将各子问题的解合并为原问题的解 }

设计划分策略, 把原问题P**分解成k个规模较小的子问题**, 这个步骤是分治算法的基础和关键, 人们往往遵循两个原则:

- **平衡子问题原则**, 分割出的k个子问题其规模最好大致相当;
- **独立子问题原则**, 分割出的k个子问题之间重叠越少越好, 最好k个子问题是相互独立, 不存在重叠子问题。

分治法的适用条件

分治法所能解决的问题一般具有以下几个特征：

- 问题的规模缩小到一定的程度就可以容易地解决

因为问题的计算复杂性一般是随着问题规模的增加而增加，因此大部分问题满足这个特征。

- 该问题可以分解为若干个规模较小的相同问题，即该问题具有最优子结构性质

这条特征是应用分治法的前提，它也是大多数问题可以满足的，此特征反映了递归思想的应用

- 利用该问题分解出的子问题的解可以合并为该问题的解；

能否利用分治法完全取决于问题是否具有这条特征，如果具备了前两条特征，而不具备第三条特征，则可以考虑贪心算法或动态规划。

- 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。

这条特征涉及到分治法的效率，如果各子问题是不独立的，则分治法要做许多不必要的工作，重复地解公共的子问题，此时虽然也可用分治法，但一般用动态规划较好。

分治法的复杂性分析

divide-and-conquer(P)

```
{  
  if ( | P | <= n0)  adhoc(P); //解决小规模的问题  
  divide P into smaller subinstances P1,P2,...,Pk;    //分解问题  
  for (i=1,i<=k,i++)  
    yi=divide-and-conquer(Pi); //递归的解各子问题  
  return merge(y1,...,yk); //将各子问题的解合并为原问题的解  
}
```

(1)分解阈值 $n_0 = 1$

(2)adhoc解规模为1的问题耗费1个单位时间

(3)将原问题分解为 k 个子问题以及用merge将 k 个子问题的解合并为原问题的解需用 $f(n)$ 个单位时间

(4) $T(n)$ 表示该分治法解规模为 $|P| = n$ 的问题所需的计算时间

$$T(n) = \begin{cases} O(1), & n = 1 \\ aT\left(\frac{n}{b}\right) + f(n), & n > 1 \end{cases}$$


$$T(n) = \begin{cases} O(1), & n = 1 \\ aT\left(\frac{n}{b}\right) + f(n), & n > 1 \end{cases} \xrightarrow{\text{假设}} \begin{cases} f(n) = O(n) \\ b = 2 \rightarrow a = 2 \end{cases}$$

$$\begin{aligned} T(n) &= aT(n/b) + f(n) \\ &= 2T(n/2) + O(n) \\ &= 2(2T(n/4) + O(n/2)) + O(n) \\ &= 4T(n/4) + 2O(n) \\ &= 8T(n/8) + 3O(n) \\ &\quad \vdots \\ &= 2^k T(n/2^k) + kO(n) \end{aligned}$$

当 $\frac{n}{2^k} = 1$ 时, 即 $k = \log n$,
得到:

$$\begin{aligned} T(n) &= nT(1) + \log n \cdot O(n) \\ &= n + O(n \log n) \\ &= O(n \log n) \end{aligned}$$

$O(n \log n)$

以下哪项不是分治法适用的条件？

- ☐ A 问题可以分解为若干个规模较小的相同问题
- ☒ B 分解出的子问题之间存在公共子问题
- ☐ C 问题的规模缩小到一定程度就可以容易地解决
- ☐ D 利用子问题的解可以合并为原问题的解

提交



二分搜索技术

给定已按升序排好序的 n 个元素 $a[0:n-1]$ ，现要在这 n 个元素中找出一特定元素。

- ✓ 该问题的规模缩小到一定的程度就可以容易地解决；

如果 $n=1$ ，即只有一个元素，则只要比较这个元素和 x 就可以确定 x 是否在表中。满足分治法的第一个适用条件

- ✓ 该问题可以分解为若干个规模较小的相同问题；
- ✓ 分解出的子问题的解可以合并为原问题的解；

比较 x 和 a 的中间元素 $a[mid]$ ，若 $x=a[mid]$ ，则 x 在 L 中的位置就是 mid ；如果 $x<a[mid]$ ，由于 a 是递增排序的， x 必然排在 $a[mid]$ 的前面；如果 $x>a[mid]$ ，同理只要在 $a[mid]$ 的后面查找 x 即可。无论是在前面还是后面查找 x ，其方法都和 a 中查找 x 一样，只不过是查找的规模缩小了。满足分治法的第二个和第三个适用条件。

- ✓ 分解出的各个子问题是相互独立的。

在 $a[i]$ 的前面或后面查找 x 是独立的子问题，因此该问题分解出的子问题相互独立，满足分治法的第四个适用条件。



二分搜索

```
int binarySearch(int[] a, int start, int end, int target) {  
    if(start > end)  
        return -1;  
    int mid = start + (end - start) / 2;  
    if(a[mid] == target)  
        return mid;  
    else if(a[mid] > target)  
        return binarySearch(a, start, mid-1, target);  
    else  
        return binarySearch(a, mid+1, end, target);  
}
```

```
int binarySearch(int[] a, int len, int target) {  
    int low = 0, high = len-1;  
    while(low <= high) {  
        int mid = low + (high - low) / 2;  
        if(a[mid] == target) return mid;  
        else if(a[mid] > target)  
            high = mid - 1;  
        else low = mid + 1;  
    }  
    return -1;  
}
```

算法复杂度分析:

每执行一次算法的while循环，待搜索数组的大小减少一半。因此，在最坏情况下，while循环被执行了 $O(\log n)$ 次。循环体内运算需要 $O(1)$ 时间，因此整个算法在最坏情况下的计算时间复杂度为 $O(\log n)$ 。



大整数的乘法

请设计一个有效的算法，可以进行两个 n 位大整数的乘法运算

◆小学的方法： $O(n^2)$

✗效率太低

◆分治法:

$$\begin{array}{l} X = \begin{array}{|c|c|} \hline \textcolor{red}{n/2\text{位}} & \textcolor{red}{n/2\text{位}} \\ \hline a & b \\ \hline \end{array} \\ Y = \begin{array}{|c|c|} \hline c & d \\ \hline \end{array} \end{array}$$

$$X = a 2^{n/2} + b \quad Y = c 2^{n/2} + d$$

$$XY = \textcolor{red}{ac} 2^n + (\textcolor{red}{ad} + \textcolor{red}{bc}) 2^{n/2} + \textcolor{red}{bd}$$

4次 $n/2$ 位整数乘法, 3次加法, 2次移位

请设计一个有效的算法，可以进行两个n位大整数的乘法运算

$$\begin{array}{l} X = \begin{array}{|c|c|} \hline a & b \\ \hline \end{array} \\ Y = \begin{array}{|c|c|} \hline c & d \\ \hline \end{array} \end{array}$$

$$X = a 2^{n/2} + b \quad Y = c 2^{n/2} + d$$

$$XY = ac 2^n + (ad+bc) 2^{n/2} + bd$$

4次 $n/2$ 位整数乘法, 3次加法 $O(n)$, 2次移位 $O(n)$

复杂度分析

$$T(n) = \begin{cases} O(1), & n = 1 \\ 4T(n/2) + O(n), & n > 1 \end{cases}$$



$$T(n) = O(n^2) \quad \text{✗没有改进}$$



$$XY = ac 2^n + (ad+bc) 2^{n/2} + bd$$

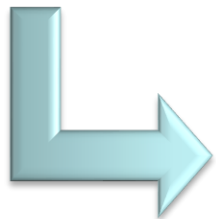
为了降低时间复杂度，必须减少乘法的次数

$$XY = ac 2^n + ((a-b)(d-c)+ac+bd) 2^{n/2} + bd$$

3次 $n/2$ 位整数乘法，6次加法 $O(n)$ ，2次移位 $O(n)$

复杂度分析

$$T(n) = \begin{cases} O(1), & n = 1 \\ 3T(n/2) + O(n), & n > 1 \end{cases}$$



$$T(n) = O(n^{\log_2 3}) = O(n^{1.59}) \quad \checkmark \text{较大的改进}$$

◆小学的方法: $O(n^2)$

✗效率太低

◆分治法: $O(n^{1.59})$

✓较大的改进

◆更快的方法??

➤如果将大整数分成更多段，用更复杂的方式把它们组合起来，将有可能得到更优的算法？

➤最终的，这个思想导致了快速傅利叶变换(Fast Fourier Transform)的产生。该方法也可以看作是一个复杂的分治算法。

➤是否能找到线性时间

□ Accelerating an FHE Integer Multiplier Using Negative Wrapped Convolution and Ping-Pong FFT

Xiang Feng; Shuguo Li

IEEE Transactions on Circuits and Systems II: Express Briefs

Year: 2019 | Volume: 66, Issue: 1 | Journal Article | Publisher: IEEE

Cited by: Papers (12)

归并排序

基本思想：将待排序元素分成大小大致**相同的2个子集合**，分别对2个子集合进行排序，最终将排好序的子集合合并成为所要求的排好序的集合。



归并排序基本思想

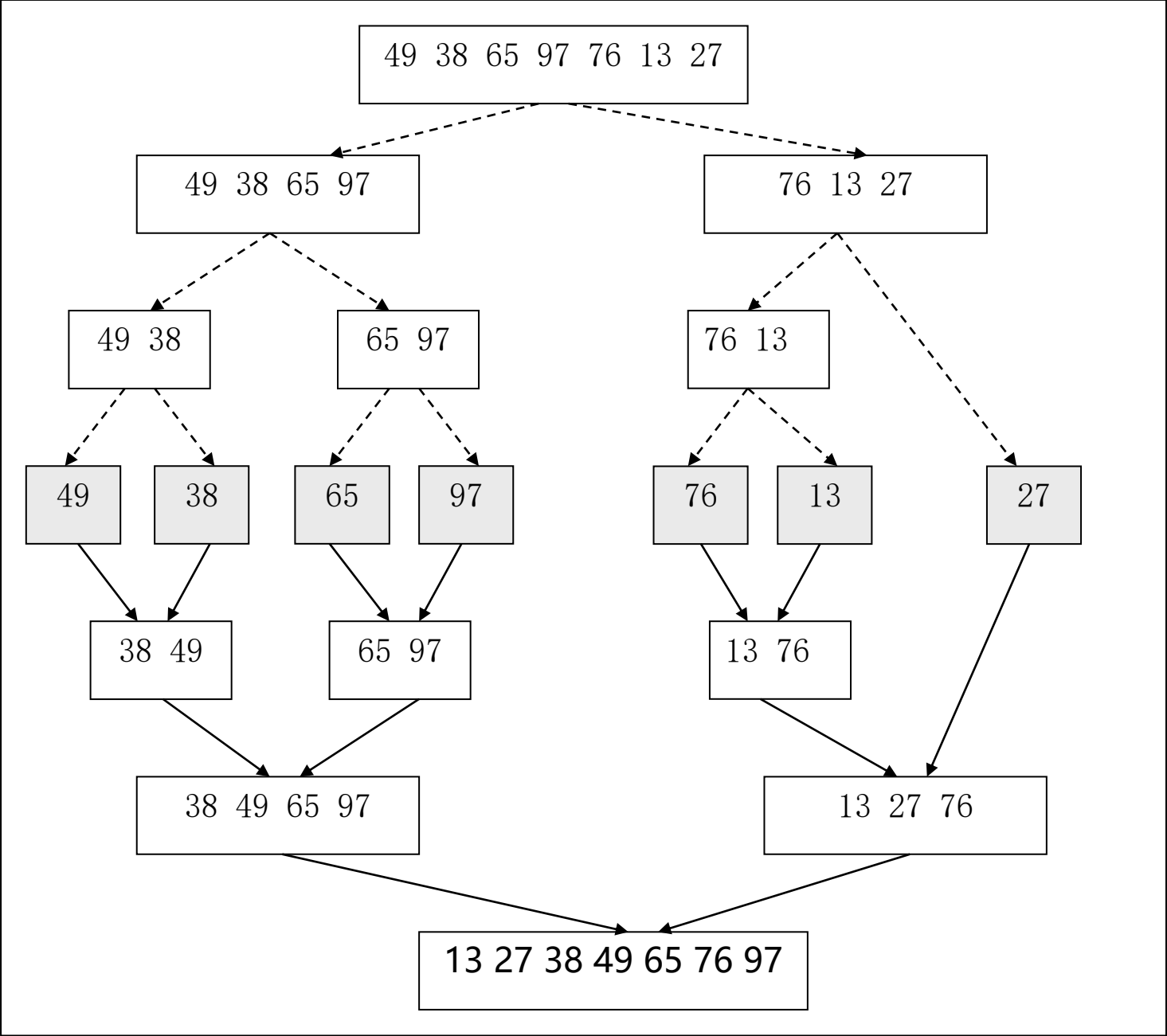
分： 根据整数集合的规模把原始的整数集合（记为 $A = \{a[l], \dots, a[r]\}$ ）平均分成两部分：
 $A_1 = \{a[l], \dots, a[(l+r)/2]\}$ 元素为第一部分， $A_2 = \{a[(l+r)/2 + 1], \dots, a[r]\}$ 为第二部分。

治： 如果划分后的子集 A_1 和 A_2 只包含一个整数，则不需任何操作，把这单个整数当成已排好序的集合；否则，把该子集继续分割，然后递归调用。

合： 设计子程序merge把两个已经升序排列的子集 B_1 ， B_2 合并成一个整体升序排列的集合 B 。

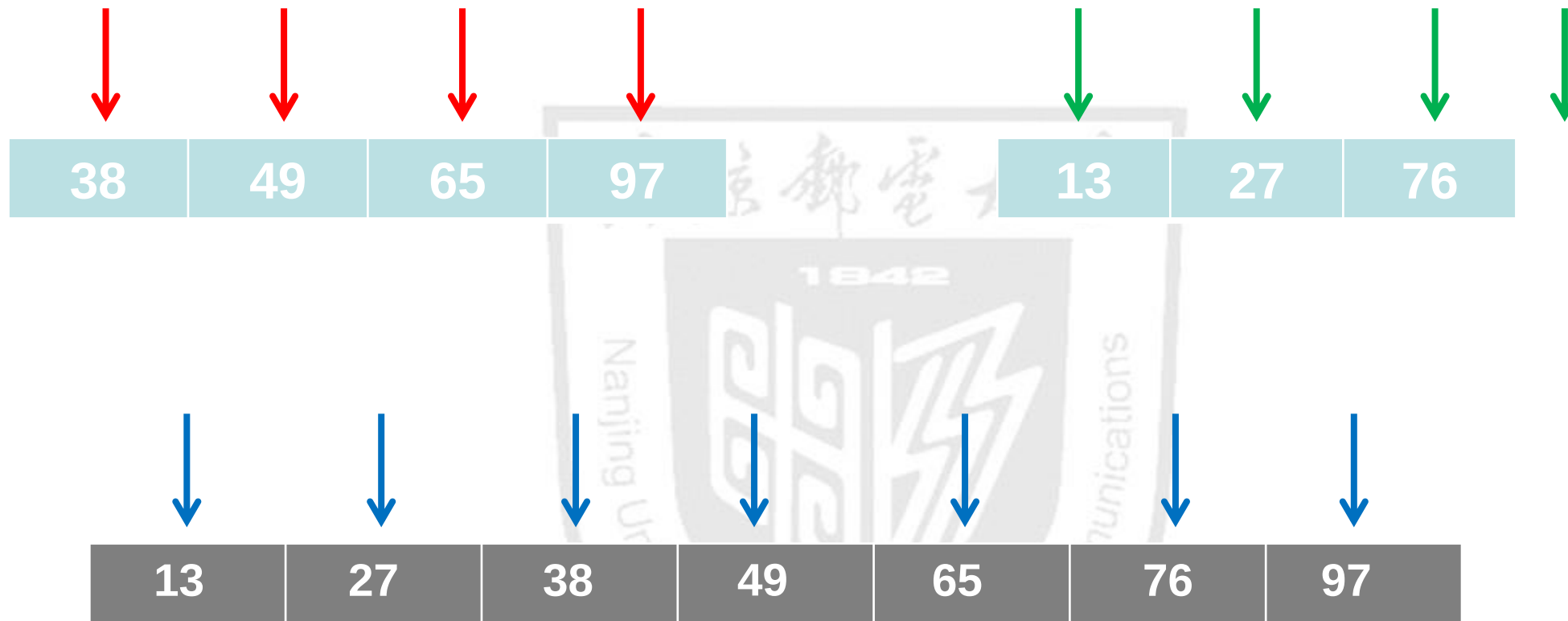


归并排序总体过程描述





归并子程序演示



归并子程序实现

$O(n)$

归并子程序 merge

需要缓存数组

// 合并iData[iLow:iMid] 与iData[iMid+1:iHigh]到iBuffer[iLow:iHigh]中

```
void merge(int iDatas[],int iBuffer[],int iLow,int iMid,int iHigh){
```

```
    int i = iLow,j = iMid+1,k = iLow;
```

```
    while((i <= iMid)&&(j <= iHigh)){
```

```
        if(iDatas[i] <= iDatas[j])
```

```
            iBuffer[k++] = iDatas[i++];
```

```
        else
```

```
            iBuffer[k++] = iDatas[j++];
```

```
    }
```

```
    if(i <= iMid) // 移动a[iLow:iMid]后面部分
```

```
        for(int ii = i; ii<= iMid; ii++) iBuffer[k++] = iDatas[ii];
```

```
    else // 移动a[iMid+1:iHigh]后面部分
```

```
        for(int jj = j; jj<= iHigh; jj++) iBuffer[k++] = iDatas[jj];
```

```
}
```

归并排序算法实现与分析

主程序:

```
void mergeSort(int iDatas[],int iBuffer[], int iLow, int iHigh){  
    if(iHigh > iLow){  
        int iMid = (iLow + iHigh)/2;  
        mergeSort(iDatas,iBuffer,iLow,iMid);  
        mergeSort(iDatas,iBuffer,iMid+1,iHigh);  
        merge(iDatas,iBuffer,iLow,iMid,iHigh);  
        for(int i=iLow;i <= iHigh;i++)  
            iDatas[i] = iBuffer[i];  
    }  
}
```

$$T(n) = \begin{cases} O(1), & n \leq 1 \\ 2T(n/2) + O(n), & n > 1 \end{cases}$$

$$T(n) = O(n \log(n))$$

练习题

- 采用归并排序算法，其中归并与复制操作的时间复杂度为 $O(n)$ ，请给出合并排序算法时间复杂度的递推公式，并证明其时间复杂度为 $O(n \log n)$ 。



(1) 时间复杂度递推公式为 $T(n) = \begin{cases} O(1), & n=1, \\ 2T(n/2) + O(n), & n>1 \end{cases}$

(2) 由递推公式可以得

$$\begin{aligned} T(n) &= 2T(n/2) + O(n) \\ &= 2(2T(n/4) + O(n/2)) + O(n) \\ &= 2^k T(n2^{-k}) + kO(n) \end{aligned}$$

当 $2n^{-k} = 1$ 时, 有 $k = \log_2 n$;

则得到时间复杂度为

$$\begin{aligned} T(n) &= nT(1) + \log_2 n O(n) \\ &= O(n \log_2 n) \end{aligned}$$

快速排序

任务描述：任意给定一包含 n 个整数的集合，把 n 个整数按升序排列。

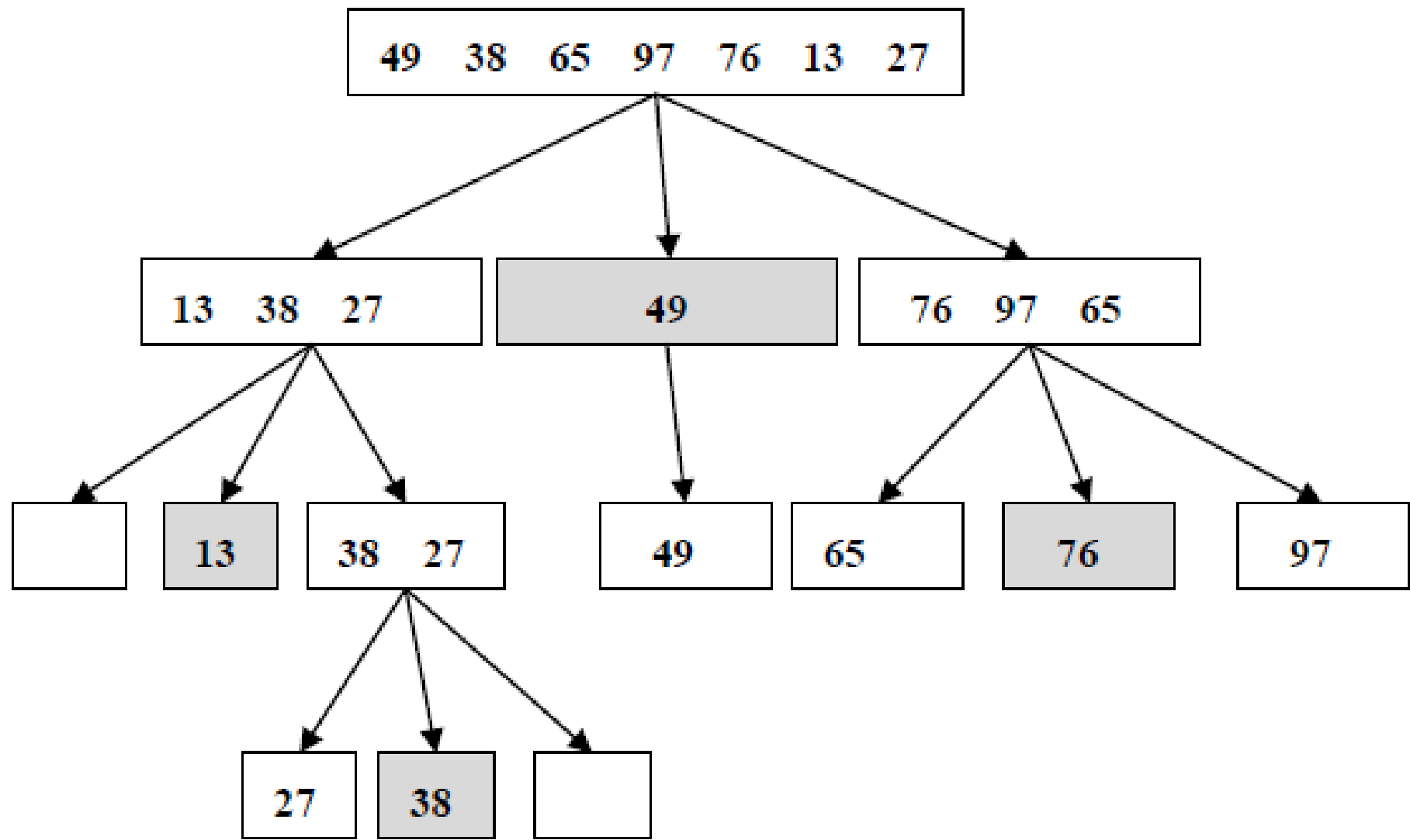
算法思路

分：首先考虑怎样对原始的整数集合（记为 $A = \{a[l], \dots, a[r]\}$ ）进行划分。以元素 $a[l]$ 为**基准元素**，把原数组 A 分为三个部分： $A_1 = \{a'[l], \dots, a'[k-1]\}$ ， $A_2 = \{a'[k]\}$ ，和 $A_3 = \{a'[k+1], \dots, a'[r]\}$ ，其中 A_1 中所有元素的值都小于等于基准元素 $a[l]$ ，而 A_3 中所有元素的值都大于基准元素 $a[l]$ ， A_2 中元素 $a'[k]$ 就是调整位置后的基准元素 $a[l]$ 。

治：如果划分后的子集 A_1 和 A_3 只包含一个整数，则不需任何操作，把这单个整数当成已排好序的集合；否则，把该子集继续分割，然后**递归调用**。子集 A_2 中元素划分后其位置保持不变。

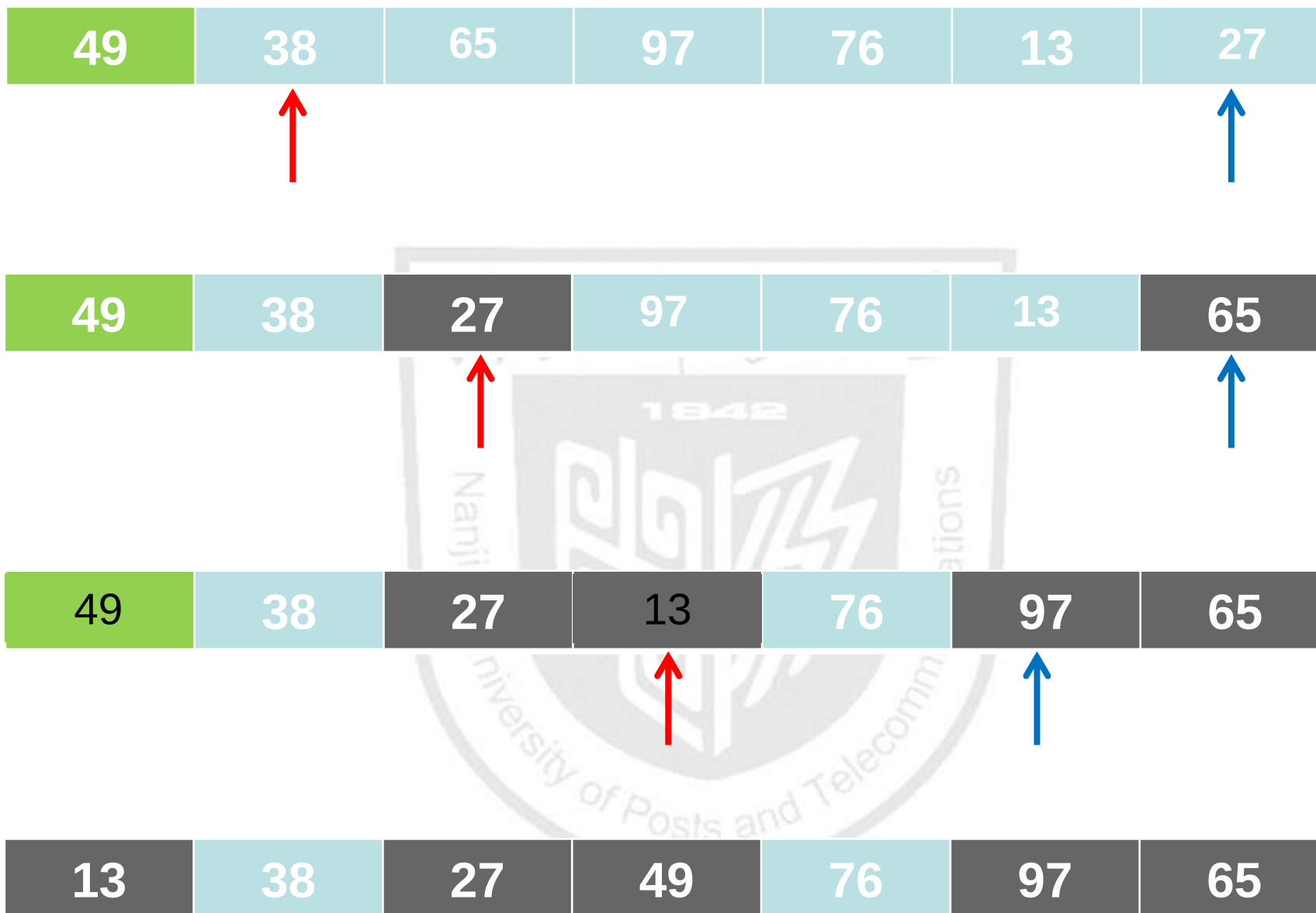
合：因为子集 A_1 和 A_3 的排序是递归调用的，所以，在 A_1 和 A_3 都已经排好序后，**不需要**执行任何操作， A_1 ， A_2 和 A_3 自然排列就是有序的。

算法过程示例





划分子程序演示



划分子程序实现

```
// 对集合iDatas[iLeft:iRight]进行划分
int partition(int iDatas[], int iLeft, int iRight){
    int i = iLeft + 1, j = iRight;
    int iAnchor = iDatas[iLeft]; // 基准元素
    while(i <= j){
        while((iDatas[i] <= iAnchor) && (i <= iRight)) i++;
        while(iDatas[j] > iAnchor) j--;
        if(i < j){
            swap(iDatas[i], iDatas[j]); // 小于等于iAnchor前移, 大于iAnchor则后移
            i++; j--;
        }
    }
    swap(iDatas[iLeft], iDatas[j]);
    return j;
}

void swap(int &iValue1, int &iValue2){
    int iTemp = iValue1;
    iValue1 = iValue2;
    iValue2 = iTemp;
}
```

$O(n)$

快速排序实现

快速排序算法主程序

```
void quickSort(int iDatas[], int iLeft, int iRight) {  
    if (iRight > iLeft) {  
        int k = partition(iDatas, iLeft, iRight);  
        quickSort(iDatas, iLeft, k-1); // 对子集合A1排序  
        quickSort(iDatas, k+1, iRight); // 对子集合A3排序  
    }  
}
```

1) 最坏情况，对于一个已经排好序的集合（比如 {1, 2, 3, 4, 5, 6}），划分为n-1和1个元素两部分：

$$T(n) = \begin{cases} O(1) & n \leq 1 \\ T(n-1) + O(n) & n > 1 \end{cases}$$

2) 最好情况，每次划分所取的基准都恰好为中值，即每次划分都产生两个大小为n/2的区域，时间复杂度为：

$$T(n) = \begin{cases} O(1) & n \leq 1 \\ 2T(n/2) + O(n) & n > 1 \end{cases}$$

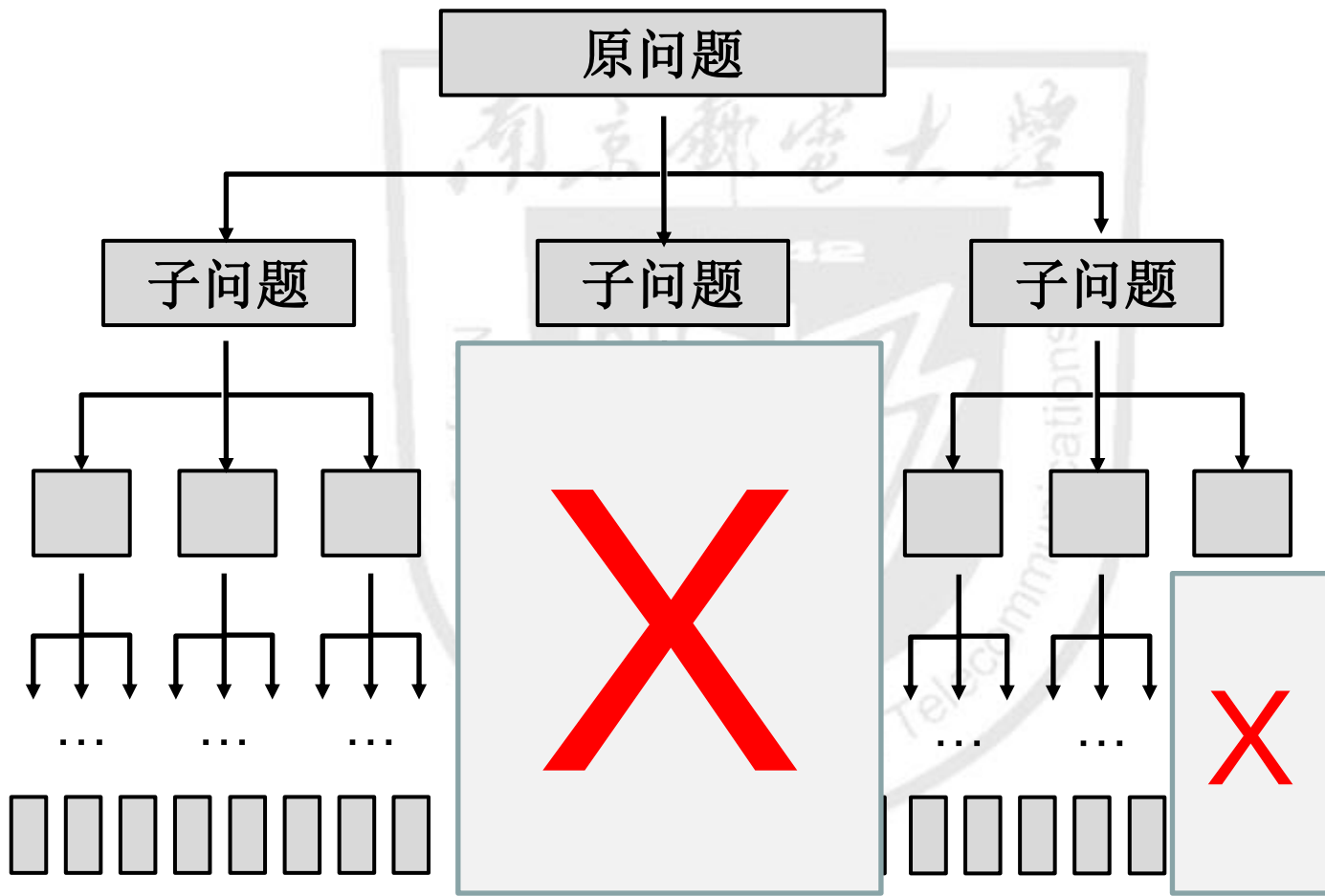


快速排序算法的性能取决于划分的对称性。通过修改算法partition，可以设计出采用随机选择策略的快速排序算法。在快速排序算法的每一步中，当数组还没有被划分时，可以在a[p:r]中随机选出一个元素作为划分基准，这样可以使划分基准的选择是随机的，从而可以期望划分是较对称的。

```
template<class Type>
int RandomizedPartition (Type a[], int p, int r)
{
    int i = Random(p,r);
    Swap(a[i], a[p]);
    return Partition (a, p, r);
}
```

减治策略基本原理

在分治算法中，如果划分的某个（或者某些）子问题与原问题的解**无关**或者与其它子问题**等同**，则这个（或者这些）子问题**不用递归求解**，**直接舍弃或者取已有结果**。



乘方运算

设a是给定的实数，计算 a^n ，其中n是自然数。

$$a^n = \begin{cases} a^{n/2} * a^{n/2} & n \% 2 = 0 \\ a * a^{(n-1)/2} * a^{(n-1)/2} & n \% 2 = 1 \\ a & n = 1 \end{cases}$$

$$T(n) = T\left(\frac{n}{2}\right) + O(1) \\ = O(\log(n))$$

```
long pow1(int a, int n){  
    if (n == 1)  
        return a;  
    if(n%2 == 0)  
        return pow1(a, n/2) * pow1(a, n/2);  
    else  
        return a * pow1(a, (n-1)/2) * pow1(a, (n-1)/2);  
}
```

$$T(n) = 2T\left(\frac{n}{2}\right) + O(1) \\ = O(n)$$

```
long pow2(int a, int n){  
    if (n == 1) return a;  
    long ans = 0;  
    if(n%2 == 0){  
        ans = pow2(a, n/2);  
        return ans * ans;  
    }else{  
        ans = pow2(a, (n-1)/2);  
        return ans * ans * a; }  
}
```

棋盘覆盖问题

在一个 $2^k \times 2^k$ 个方格组成的棋盘中，恰有一个方格与其他方格不同，称该方格为**特殊方格**，且称该棋盘为**特殊棋盘**。在棋盘覆盖问题中，要用图示的4种不同形态的**L型骨牌**覆盖给定的特殊棋盘上除特殊方格以外的所有方格，且任何2个L型骨牌不得重叠覆盖。

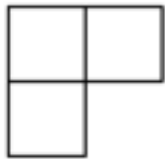
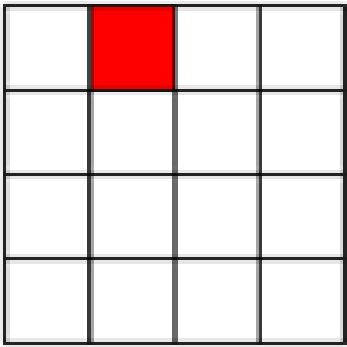
输入： 给定 $k(1 \leq k \leq 10)$ ，表示棋盘大小为 $2^k \times 2^k$ ，给出特殊方格坐标为 (x,y) ， $0 \leq x, y \leq 1024$ 。

输出： 一个边长为 2^k 的方阵，特殊方格编号0，其它方格输出L型骨牌编号（L型骨牌编号依次为1,2,3,4），数据间间隔一个Tab符号。

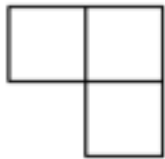
样例输入： 2 0 1

样例输出：

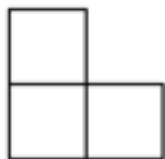
3	0	2	2
3	3	4	2
3	4	4	4
3	3	4	4



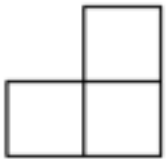
1



2



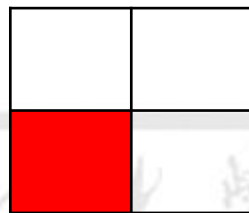
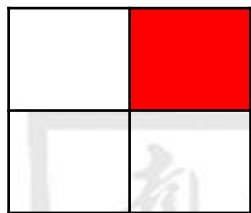
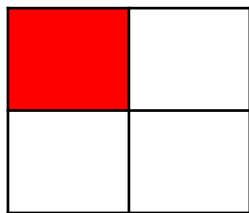
3



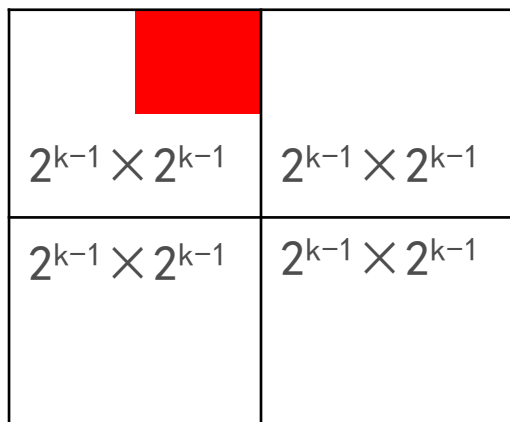
4

棋盘覆盖问题

1) 当 $k = 1$ 时，可以容易选择一个L型骨牌覆盖

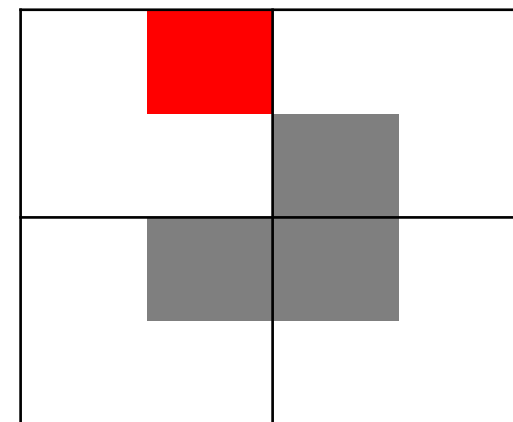


2) 当 $k > 1$ 时，可以从中间分为4个子棋盘

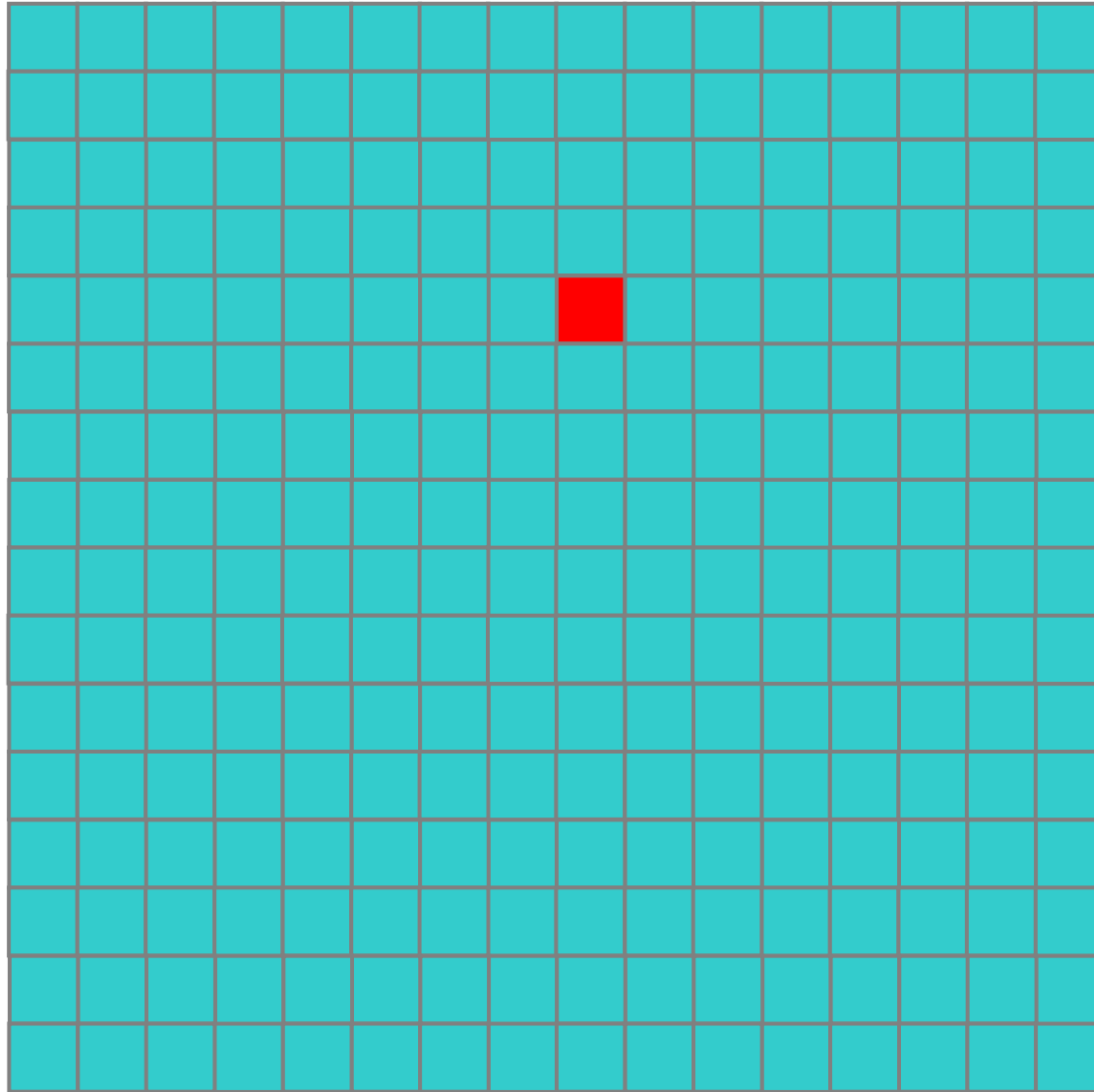


- 特殊方格必位于4个子棋盘之一，其余3个子棋盘中**无特殊方格!!!**
- 不是相同性质的子问题

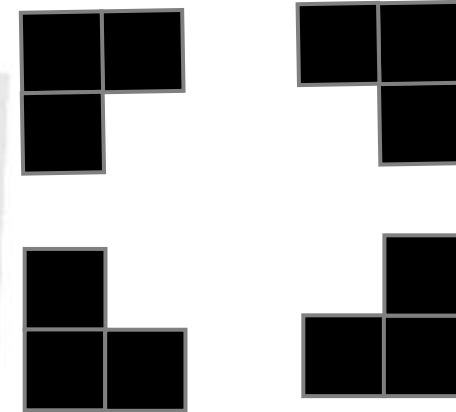
L型骨牌覆盖这3个较小棋盘的结合处



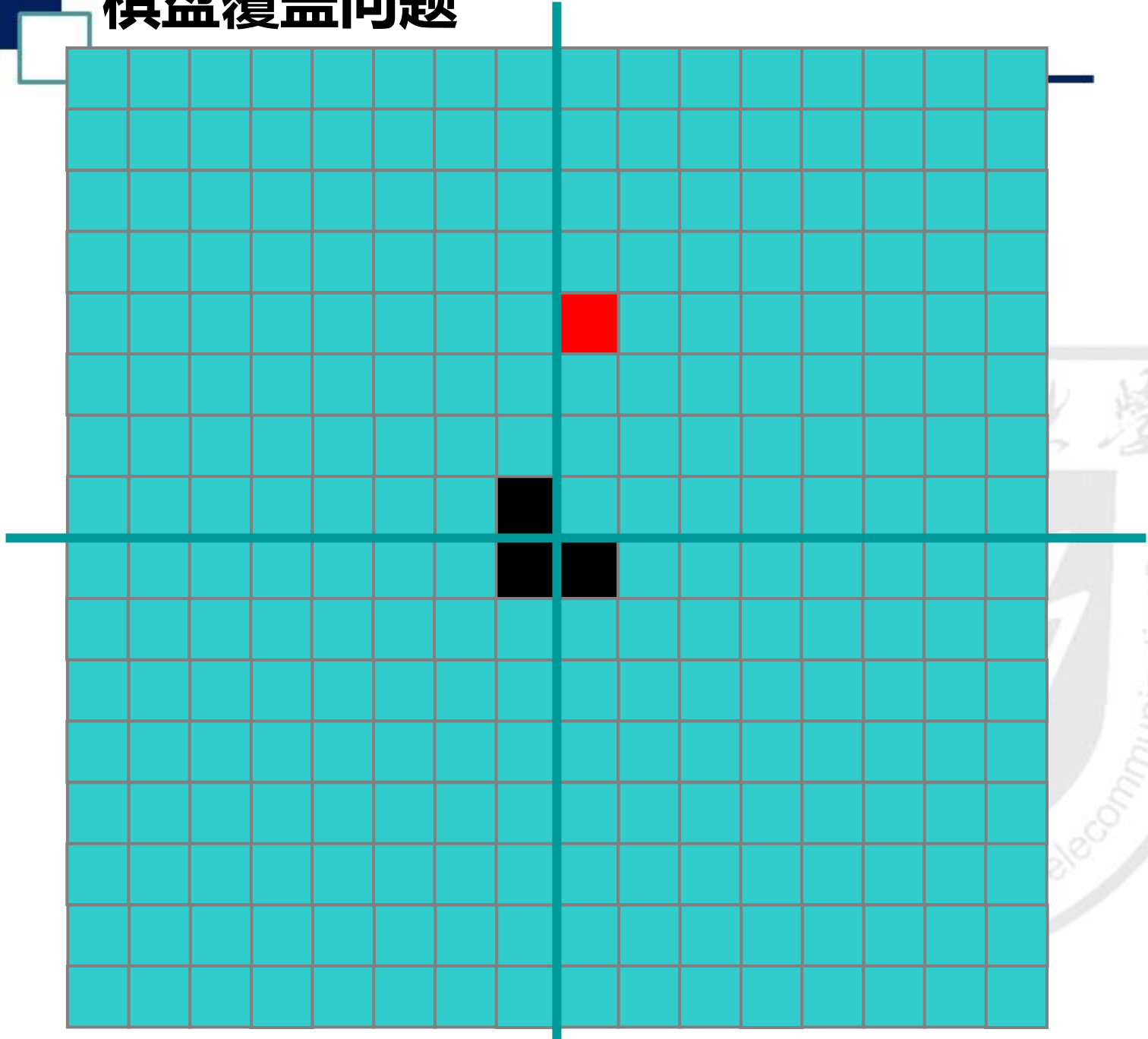
棋盘覆盖问题



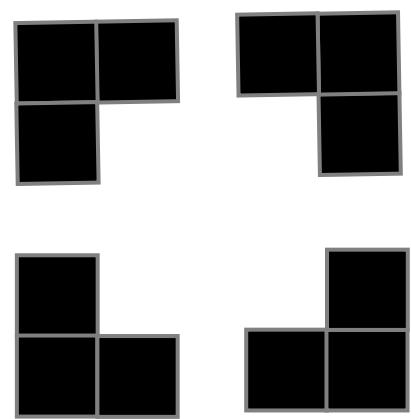
过程示例：



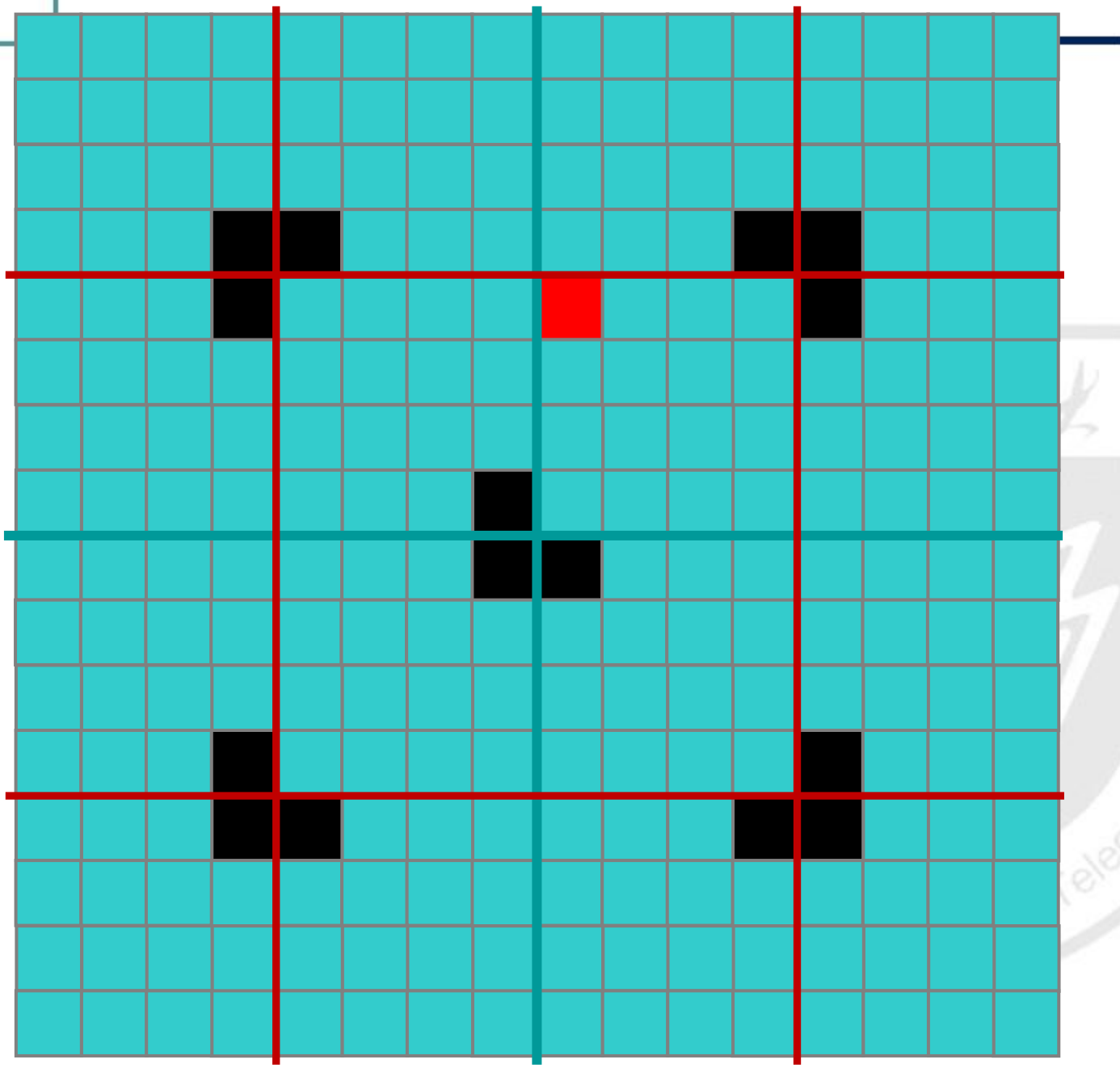
棋盘覆盖问题



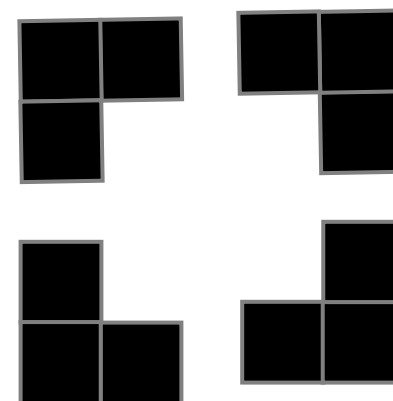
第一次分割



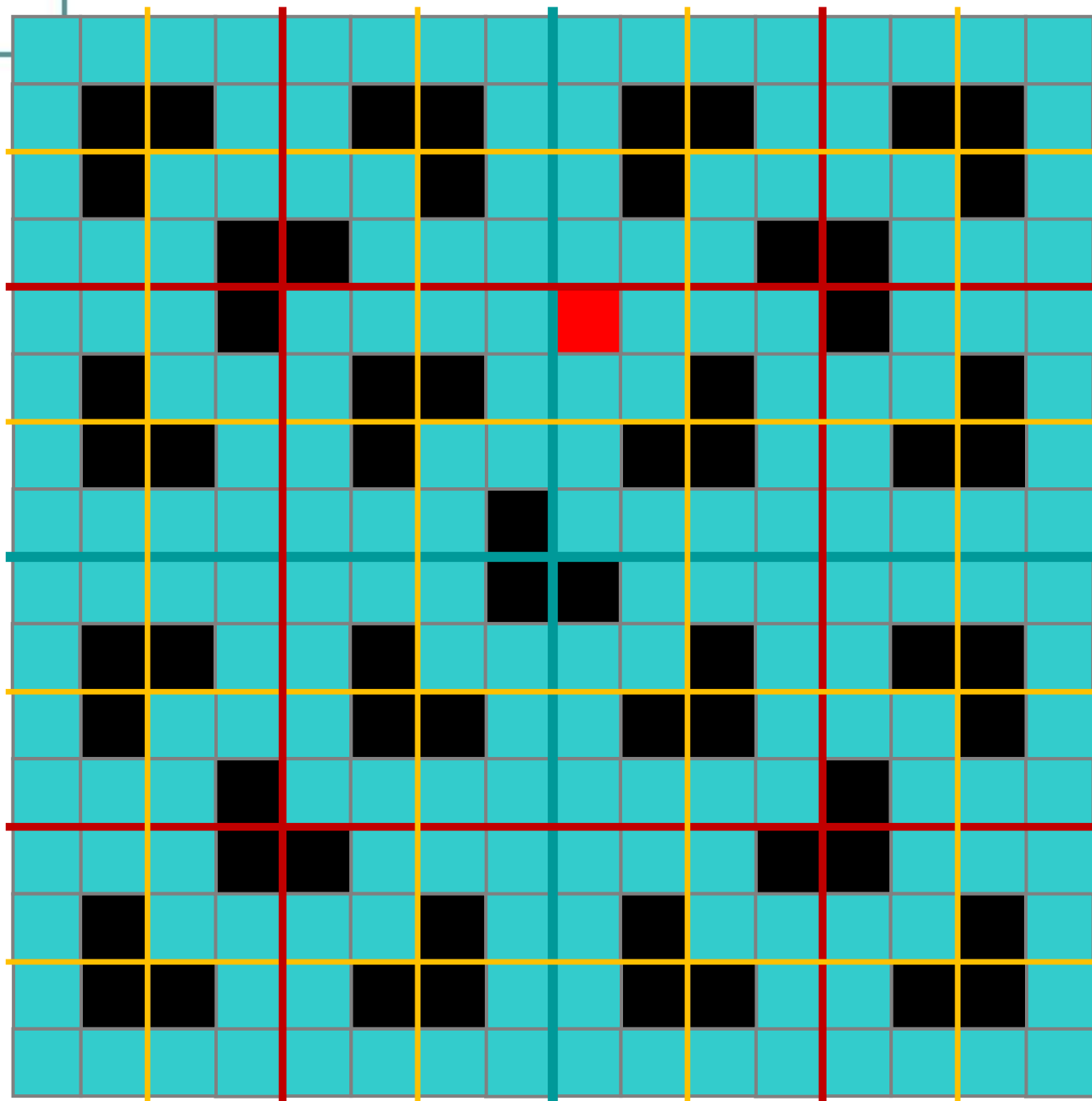
棋盘覆盖问题



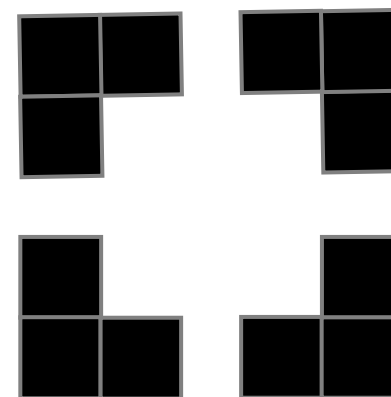
第二次分割



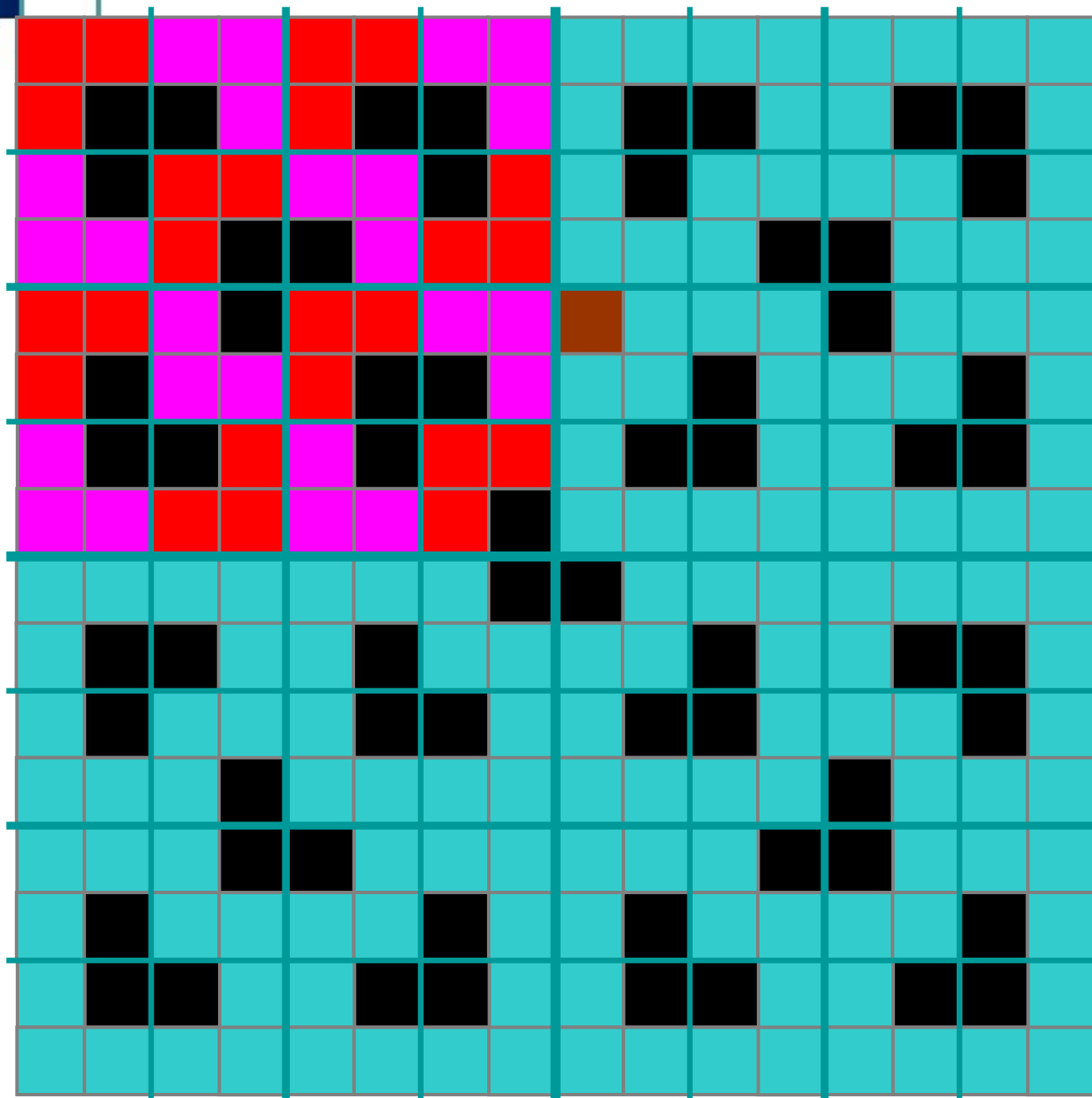
棋盘覆盖问题



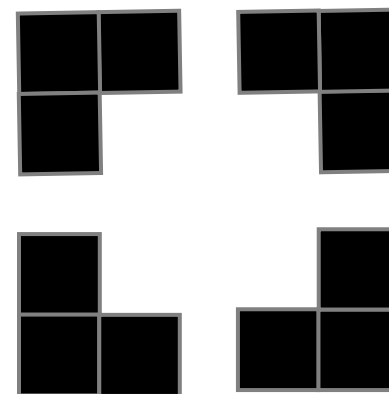
第三次分割



棋盘覆盖问题



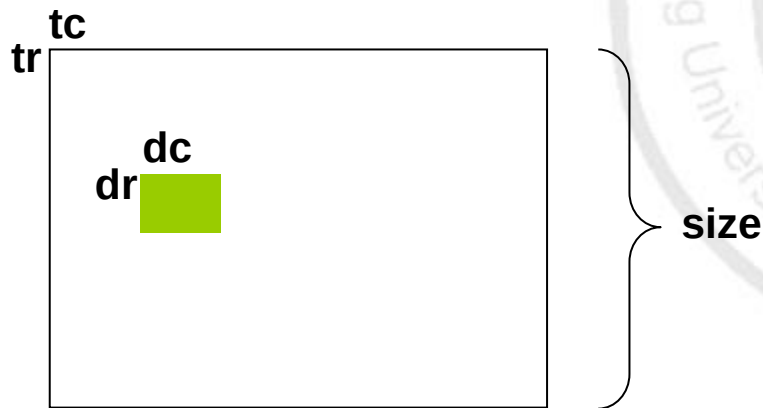
基础子问题覆盖



棋盘覆盖问题

实现细节：

- ① 棋盘表示：用二维数组 $\text{Board}[\text{size}][\text{size}]$ 表示一个棋盘， $\text{Board}[0][0]$ 是棋盘的左上角方格。其中， $\text{size}=2^k$ 。为了在递归处理的过程中使用同一个棋盘，将数组 Board 设为全局变量；
- ② 子棋盘表示：在棋盘数组 $\text{Board}[\text{size}][\text{size}]$ 中，由子棋盘左上角的下标 tr 、 tc 和棋盘边长 s 表示；
- ③ 特殊方格：用 $\text{Board}[\text{dr}][\text{dc}]$ 表示， dr 和 dc 是该特殊方格在棋盘数组 Board 中的下标；
- ④ 输出：把输入棋盘结合处的3个方格用对应的L型棋盘覆盖，即对应位置赋值为相应L型棋盘编号。



算法输入参数：

- tr : 棋盘左上角方格的行号
- tc : 棋盘左上角方格的列号
- dr : 特殊方格所在的行号
- dc : 特殊方格所在的列号
- size : $\text{size} = 2^k$

棋盘覆盖问题

```
void ChessBoard(int tr, int tc, int dr, int dc, int size)
{
```

```
    if (size == 2)
        fill(tr, tc, dr, dc); //覆盖2×2的棋盘
```

```
    s = size/2; // 划分棋盘
```

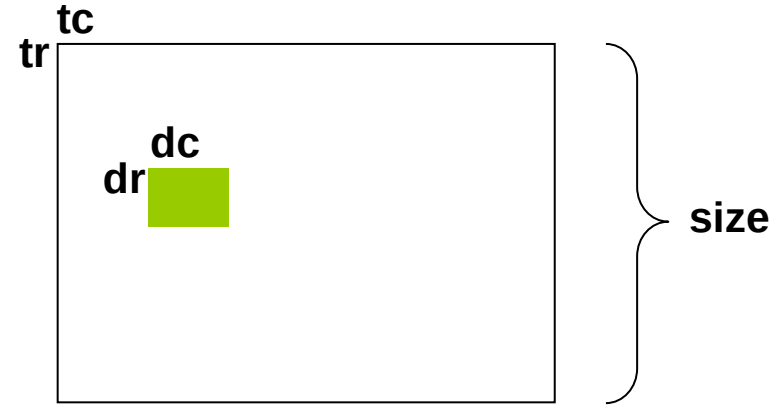
```
    if (dr < tr + s && dc < tc + s) { // 特殊方格在左上角子棋盘中
        ChessBoard(tr, tc, dr, dc, s); //递归处理左 upper 子棋盘
        ChessBoard(tr+s, tc, tr+s, tc+s-1, s); //递归处理左 lower 子棋盘
        ChessBoard(tr, tc+s, tr+s-1, tc+s+1, s); //递归处理右 upper 子棋盘
        ChessBoard(tr+s, tc+s, tr+s+1, tc+s+1, s); //递归处理右 lower 子棋盘
    }
```

```
    if (dr < tr + s && dc >= tc + s) { ..... } // 特殊方格在右上角子棋盘中
```

```
    if (dr >= tr + s && dc < tc + s) { ..... } // 特殊方格在左下角子棋盘中
```

```
    if (dr >= tr + s && dc >= tc + s) { ..... } // 特殊方格在右下角子棋盘中
```

```
}
```



$$T(k) = \begin{cases} O(1) & k=0 \\ 4T(k-1) + O(1) & k>0 \end{cases}$$

$O(4^n)$



谢谢 THANKS

